

UNIVERSITÀ TELEMATICA e-Campus

Corso di Laurea in

INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE (DM 1648/23)

Tesi di Laurea

**Sicurezza nei sistemi Internet of Things: architetture,
vulnerabilità e strategie di mitigazione**

Relatore: Prof. Oleksandr Kuznetsov

Candidato: Samuele Marchitelli

Matricola: 001814763

Anno Accademico 2025/2026

AUTORIZZAZIONE ALLA CONSULTAZIONE DELLA TESI DI LAUREA

Il sottoscritto Samuele Marchitelli, matricola n. 001814763,
autore della tesi dal titolo:

“Sicurezza nei sistemi Internet of Things: architetture, vulnerabilità e strategie di mitigazione”

☒ AUTORIZZA

☐ NON AUTORIZZA

la consultazione della tesi, fatto divieto di riprodurre parzialmente o integralmente il contenuto senza corretta attribuzione.

Dichiara inoltre di:

☒ AUTORIZZARE

☐ NON AUTORIZZARE

l'Università Telematica eCampus, nei limiti previsti dalla normativa vigente, al trattamento, alla comunicazione, alla diffusione e alla pubblicazione dei dati personali per le finalità istituzionali connesse alla tesi di laurea.

Data: 26/08/2026

Firma: _____

Abstract

La crescente diffusione dei sistemi Internet of Things amplia la superficie di attacco delle reti, nelle quali dispositivi con risorse limitate, configurazioni deboli e aggiornamenti discontinui possono diventare punti di ingresso per attività malevole. Questa tesi analizza l'impiego di Intrusion Detection Systems basati su machine learning e sviluppa una pipeline riproducibile per la classificazione del traffico IoT sul dataset TON_IoT/UNSW, utilizzando il file `train_test_network.csv` in modalità binaria e multiclasse.

La metodologia comprende validazione dei dati, preprocessing senza contaminazione tra training e test set, codifica delle variabili categoriche, normalizzazione, split stratificato e valutazione mediante accuracy, precision, recall, F1-score, Macro F1 e Weighted F1. La baseline confronta Random Forest, LightGBM, XGBoost e Logistic Regression. Gli esperimenti estesi includono SMOTE, tuning iperparametrico, MLPClassifier, Isolation Forest, analisi SHAP, misure di latenza e dimensione degli artefatti, compressione e un dimostratore embedded simulato su ESP32 tramite Wokwi.

I risultati mostrano prestazioni particolarmente elevate per i modelli ensemble, con LightGBM migliore nella baseline binaria e multiclasse. Logistic Regression presenta prestazioni inferiori, ma offre semplicità, bassa latenza e maggiore facilità di portabilità. Il confronto tra modello Python e coefficienti quantizzati chiarisce il costo della semplificazione richiesta dal deployment embedded. La tesi collega infine i risultati a strategie di mitigazione quali segmentazione, hardening, aggiornamenti firmware, autenticazione forte, inventario degli asset, logging e principi zero trust. Il lavoro costituisce una base sperimentale completa e riproducibile; traffico live, drift e validazione su hardware fisico restano sviluppi futuri.

Indice

Elenco delle figure	vii
Elenco delle tabelle	viii
Capitolo 1 - Introduzione	1
1.1 Contesto dei sistemi Internet of Things	1
1.2 Crescita degli ambienti connessi e aumento della superficie di attacco	1
1.3 Problema della sicurezza IoT	2
1.4 Obiettivo e contributo della tesi	3
1.5 Struttura dell'elaborato	3
Capitolo 2 - Stato dell'arte	4
2.1 Architetture IoT	4
2.2 Protocolli e comunicazione	4
2.3 Vulnerabilità tipiche	5
2.4 Attacchi in ambienti IoT	5
2.5 Intrusion Detection Systems	6
2.6 Machine learning applicato agli IDS	6
2.7 Requisiti di sicurezza in un ambiente IoT	7
2.8 IDS in architetture edge e cloud	7
2.9 Difficoltà specifiche del machine learning per la sicurezza	8
Capitolo 3 - Metodologia	9
3.1 Organizzazione del repository	9
3.2 Dataset utilizzato	9
3.3 Validazione dei dati	10
3.4 Preprocessing e prevenzione del data leakage	10
3.5 Split sperimentale	12
3.6 Modelli baseline	12
3.7 Esperimenti estesi	12

3.8 Metriche di valutazione	13
3.9 Protocollo di esecuzione	13
3.10 Gestione degli artefatti.....	14
3.11 Simulazione IoT e dimostrazione embedded	14
Capitolo 4 - Risultati sperimentali	15
4.1 Premessa sui risultati	15
4.2 Risultati baseline nella classificazione binaria	15
4.3 Risultati baseline nella classificazione multiclasse	17
4.4 Effetto del bilanciamento tramite SMOTE	20
4.5 Tuning iperparametrico.....	21
4.6 Modelli aggiuntivi: MLPClassifier e Isolation Forest	22
4.7 Interpretabilità dei modelli.....	24
4.8 Latenza, dimensione e compressione.....	26
4.9 Dimostratore embedded con Wokwi.....	27
4.10 Sintesi degli artefatti prodotti	28
4.11 Lettura critica dei risultati	28
Capitolo 5 - Discussione e strategie di mitigazione.....	29
5.1 Interpretazione complessiva dei risultati	29
5.2 Interpretabilità contro prestazioni	29
5.3 Limiti del dataset statico	30
5.4 Limiti della simulazione Wokwi.....	30
5.5 Strategie di mitigazione	30
5.6 Domande tecniche prevedibili	31
5.7 Possibile percorso verso una validazione su hardware reale	32
5.8 Indicazioni operative per un IDS IoT	32
5.9 Considerazioni etiche e di gestione dei dati.....	33
Capitolo 6 - Conclusioni	34

Bibliografia	35
--------------------	----

Elenco delle figure

Figura 1 - Distribuzione delle classi binarie prima del preprocessing.....	11
Figura 2 - Distribuzione delle classi binarie dopo il preprocessing.....	11
Figura 3 - Confronto dell'accuracy dei modelli baseline nella classificazione binaria...	16
Figura 4 - Confronto del Macro F1 dei modelli baseline nella classificazione binaria ..	16
Figura 5 - Matrice di confusione LightGBM nella classificazione binaria	17
Figura 6 - Confronto dell'accuracy dei modelli baseline nella classificazione multiclasse	18
Figura 7 - Confronto del Macro F1 dei modelli baseline nella classificazione multiclasse	19
Figura 8 - Matrice di confusione LightGBM nella classificazione multiclasse	20
Figura 9 - Confronto Macro F1 con e senza SMOTE.....	21
Figura 10 - Confronto Macro F1 dei modelli sottoposti a tuning binario.....	22
Figura 11 - Confronto Macro F1 degli esperimenti estesi binari.....	23
Figura 12 - Confronto Macro F1 degli esperimenti estesi multiclasse	23
Figura 13 - Importanza delle feature nel modello LightGBM binario	25
Figura 14 - Importanza delle feature nel modello LightGBM multiclasse.....	26

Elenco delle tabelle

Tabella 1 - Prestazioni dei modelli baseline nella classificazione binaria	15
Tabella 2 - Prestazioni dei modelli baseline nella classificazione multiclasse	17

Capitolo 1 - Introduzione

1.1 Contesto dei sistemi Internet of Things

Con l'espressione Internet of Things si indica un insieme di dispositivi fisici connessi in rete, capaci di raccogliere dati dall'ambiente, elaborarli localmente o trasmetterli verso altri sistemi. A differenza dei computer tradizionali, molti dispositivi IoT sono progettati per svolgere funzioni specifiche: misurare temperatura, umidità, pressione, vibrazioni, consumo energetico, posizione, movimento, apertura di porte, qualità dell'aria o parametri industriali. La loro utilità nasce dalla capacità di trasformare fenomeni fisici in dati digitali utilizzabili da applicazioni, dashboard e sistemi decisionali.

Gli scenari applicativi sono numerosi. Nelle abitazioni intelligenti, sensori e attuatori gestiscono illuminazione, climatizzazione, videosorveglianza e sicurezza domestica. Nelle smart city, dispositivi distribuiti supportano monitoraggio del traffico, illuminazione pubblica, parcheggi intelligenti e gestione ambientale. Nel settore sanitario, dispositivi connessi possono raccogliere dati clinici e supportare il monitoraggio remoto. Nell'industria, l'Industrial Internet of Things consente manutenzione predittiva, controllo di processo e raccolta di dati macchina. In ciascuno di questi casi, l'affidabilità del sistema dipende non solo dalla correttezza funzionale, ma anche dalla sicurezza delle comunicazioni e dei dispositivi.

L'elemento che rende gli ambienti IoT particolarmente complessi è l'eterogeneità. Una stessa infrastruttura può includere microcontrollori, dispositivi embedded Linux, gateway industriali, reti wireless, broker MQTT, applicazioni cloud, API, database e interfacce web. Ogni componente utilizza tecnologie, protocolli e cicli di aggiornamento differenti. Di conseguenza, la sicurezza IoT non può essere affrontata come un problema isolato del singolo dispositivo: deve essere analizzata a livello architetturale, considerando l'intero percorso del dato, dalla raccolta alla trasmissione, dall'elaborazione alla conservazione [15].

1.2 Crescita degli ambienti connessi e aumento della superficie di attacco

La crescita degli ambienti connessi ha aumentato il numero di punti potenzialmente vulnerabili. Ogni dispositivo collegato alla rete rappresenta un possibile ingresso per un attaccante, soprattutto quando utilizza credenziali deboli, firmware obsoleto, servizi di rete esposti o configurazioni predefinite. Il problema è aggravato dal fatto che molti

dispositivi IoT vengono installati e poi dimenticati: continuano a funzionare per anni senza aggiornamenti regolari, senza monitoraggio e senza una gestione centralizzata degli asset.

In una rete tradizionale, workstation e server sono spesso amministrati con strumenti consolidati di sicurezza, logging, patch management e controllo accessi. In una rete IoT, invece, i dispositivi possono essere poco visibili agli amministratori, difficili da aggiornare e privi di agent di sicurezza installabili localmente. Inoltre, alcune reti IoT includono dispositivi economici, con memoria e CPU limitate, nei quali l'esecuzione di algoritmi crittografici complessi o sistemi di monitoraggio avanzati può risultare non banale.

La conseguenza è che la superficie di attacco non cresce solo in termini quantitativi, ma anche qualitativi. Un attaccante può sfruttare un dispositivo IoT compromesso come punto di appoggio per muoversi lateralmente nella rete, come nodo di una botnet, come sorgente di traffico malevolo o come mezzo per alterare dati fisici. In contesti industriali, la compromissione può incidere sulla continuità operativa; in contesti sanitari, può avere impatti sulla riservatezza e sulla sicurezza del paziente; in contesti urbani, può interferire con servizi pubblici.

1.3 Problema della sicurezza IoT

La sicurezza IoT presenta difficoltà specifiche. In primo luogo, molti dispositivi sono progettati privilegiando costo, autonomia energetica e facilità di installazione. Questi obiettivi sono legittimi dal punto di vista commerciale, ma possono portare a trascurare aspetti come autenticazione forte, aggiornamenti sicuri, cifratura del traffico, logging e protezione delle credenziali. In secondo luogo, la lunga vita operativa di alcuni dispositivi rende difficile mantenere un livello di sicurezza coerente nel tempo.

Un ulteriore problema riguarda la visibilità. Per proteggere una rete è necessario sapere quali dispositivi sono presenti, quali servizi espongono, quali protocolli utilizzano e quale comportamento di traffico è normale. Negli ambienti IoT questa informazione non è sempre disponibile. Dispositivi installati in momenti diversi, da fornitori diversi e con finalità diverse possono convivere nella stessa rete senza una governance uniforme.

La sicurezza deve quindi combinare misure preventive e misure di rilevamento. Le prime includono hardening, configurazioni sicure, segmentazione, autenticazione, aggiornamenti e controllo accessi. Le seconde includono monitoraggio, logging, sistemi

IDS e analisi del traffico. Gli Intrusion Detection Systems sono particolarmente importanti perché consentono di individuare attività sospette anche quando le misure preventive non sono sufficienti a impedire la compromissione [3], [17].

1.4 Obiettivo e contributo della tesi

L'obiettivo della tesi è progettare, implementare e documentare una pipeline sperimentale per il rilevamento di intrusioni in ambienti IoT mediante modelli di machine learning. Il lavoro parte dal repository di riferimento indicato dal relatore, ne analizza la struttura e ne riproduce la pipeline principale, introducendo al tempo stesso una riorganizzazione del codice e una documentazione più adatta a un progetto di tesi.

Il contributo sperimentale è articolato in più livelli. Il primo livello consiste nella riproduzione della baseline: preprocessing del dataset TON_IoT, classificazione binaria, classificazione multiclasse, training dei modelli principali e generazione delle metriche. Il secondo livello introduce estensioni sperimentali: SMOTE, tuning iperparametrico, MLPClassifier e Isolation Forest. Il terzo livello riguarda aspetti di interpretabilità e deployment, con feature importance, misure di latenza, compressione degli artefatti e dimostrazione embedded simulata in Wokwi.

La finalità non è proporre un IDS produttivo pronto per essere installato in una rete reale, ma costruire una base sperimentale riproducibile e tecnicamente verificabile. Ogni metrica riportata deriva dall'esecuzione del codice e viene tracciata nei file CSV prodotti dagli script. Il repository è organizzato per permettere al relatore o a un altro studente di ripetere gli esperimenti, verificare le scelte implementative e proseguire il lavoro.

1.5 Struttura dell'elaborato

Il Capitolo 2 presenta lo stato dell'arte relativo ad architetture IoT, vulnerabilità, attacchi e sistemi IDS. Il Capitolo 3 descrive la metodologia sperimentale, il dataset, il preprocessing, i modelli e il protocollo di valutazione. Il Capitolo 4 riporta i risultati ottenuti, includendo tabelle e figure generate dalla pipeline. Il Capitolo 5 discute i risultati, i limiti sperimentali e le strategie di mitigazione. Il Capitolo 6 conclude il lavoro e individua possibili sviluppi futuri.

Capitolo 2 - Stato dell'arte

2.1 Architetture IoT

Le architetture IoT sono spesso rappresentate mediante una suddivisione a livelli. Il livello percettivo include sensori e attuatori che interagiscono direttamente con l'ambiente fisico. Il livello di comunicazione comprende protocolli e tecnologie di rete che consentono ai dispositivi di trasmettere dati. Il livello edge o gateway aggrega, filtra o pre-elabora le informazioni. Il livello applicativo, infine, comprende piattaforme cloud, dashboard, sistemi di analisi e applicazioni finali [15].

Questa suddivisione è utile perché evidenzia come la sicurezza debba essere applicata lungo l'intera catena. Un sensore può essere fisicamente manomesso, un canale wireless può essere intercettato, un gateway può essere compromesso, un broker MQTT può essere configurato senza autenticazione, un'API cloud può esporre dati sensibili. Il rischio complessivo dipende dalla somma di queste esposizioni e dalla capacità dell'architettura di limitarne gli effetti.

Nel contesto IoT, un elemento centrale è il gateway. Il gateway può svolgere funzioni di traduzione di protocollo, filtraggio, buffering, aggregazione e inoltrare verso il cloud. Collocare funzioni IDS a livello di gateway può essere vantaggioso perché il gateway dispone di maggiore capacità computazionale rispetto ai sensori e ha visibilità su più flussi di traffico. Tuttavia, se il gateway diventa un punto singolo di compromissione, la sua protezione diventa critica.

2.2 Protocolli e comunicazione

I sistemi IoT possono utilizzare protocolli diversi a seconda dei requisiti di consumo energetico, portata, latenza e affidabilità. Wi-Fi ed Ethernet sono comuni in ambienti domestici e industriali; Bluetooth Low Energy è usato per dispositivi a corto raggio; Zigbee e Thread sono diffusi in reti mesh; LoRaWAN è adatto a comunicazioni a lunga distanza con basso consumo; MQTT è ampiamente usato come protocollo applicativo publish/subscribe.

MQTT è particolarmente rilevante perché consente a dispositivi leggeri di pubblicare messaggi su topic gestiti da un broker. Il modello publish/subscribe semplifica la comunicazione, ma introduce rischi se il broker è esposto senza autenticazione, senza TLS o con autorizzazioni troppo permissive. Un attaccante potrebbe pubblicare

messaggi falsi, iscriversi a topic sensibili o generare traffico anomalo. Per questo motivo, la sicurezza del broker e delle credenziali è un elemento importante nelle architetture IoT [4].

2.3 Vulnerabilità tipiche

Le vulnerabilità più frequenti nei sistemi IoT includono credenziali deboli o predefinite, servizi non necessari attivi, firmware non aggiornato, assenza di cifratura, gestione inadeguata delle chiavi, API non protette, configurazioni cloud errate e mancanza di segmentazione. Queste vulnerabilità non sono isolate: spesso si combinano tra loro, generando catene di attacco [3], [15], [17].

Un esempio tipico è un dispositivo esposto in rete con credenziali predefinite. Dopo l'accesso, l'attaccante può modificare configurazioni, installare codice malevolo o usare il dispositivo per generare traffico verso altri target. Se la rete non è segmentata, il dispositivo compromesso può diventare un ponte verso sistemi più critici. Se non esiste logging centralizzato, l'attacco può rimanere inosservato.

La presenza di firmware obsoleto rappresenta un ulteriore rischio. Anche quando una vulnerabilità è nota e corretta dal produttore, molti dispositivi rimangono non aggiornati per mancanza di procedure automatiche, per timore di interrompere il servizio o per assenza di manutenzione. In ambienti industriali, la necessità di continuità operativa rende talvolta complesso applicare patch con rapidità.

2.4 Attacchi in ambienti IoT

Gli attacchi in ambienti IoT possono assumere forme diverse. Gli attacchi di scanning cercano dispositivi e servizi esposti. Gli attacchi di brute force tentano credenziali deboli. Gli attacchi DoS o DDoS mirano a compromettere la disponibilità. Gli attacchi di spoofing e man-in-the-middle alterano o intercettano comunicazioni. Le botnet IoT sfruttano dispositivi compromessi per generare traffico coordinato.

Dal punto di vista IDS, questi attacchi possono manifestarsi attraverso pattern di traffico osservabili: aumento del numero di connessioni, variazione delle porte di destinazione, volumi anomali di byte, sequenze insolite di pacchetti, durate anomale dei flussi o cambiamenti nella distribuzione delle classi di traffico. Per questo motivo, dataset di rete come TON_IoT sono utili per addestrare e confrontare modelli di classificazione.

2.5 Intrusion Detection Systems

Un Intrusion Detection System ha il compito di rilevare attività sospette o malevole. Gli IDS possono essere host-based, quando analizzano eventi su un singolo dispositivo, oppure network-based, quando osservano il traffico di rete. In ambito IoT, la collocazione dell'IDS è una scelta progettuale importante: installarlo direttamente sul dispositivo può ridurre la latenza ma richiede risorse; installarlo su gateway o edge node offre maggiore capacità computazionale; installarlo in cloud consente analisi centralizzate ma introduce dipendenza dalla connettività [16].

Gli IDS basati su firme confrontano il traffico con regole note. Sono efficaci per attacchi già conosciuti, ma limitati rispetto a varianti nuove. Gli IDS basati su anomalie modellano il comportamento normale e segnalano deviazioni. Questa seconda categoria è adatta agli ambienti IoT perché molti dispositivi hanno comportamenti ripetitivi: un sensore invia misurazioni periodiche, un attuatore riceve comandi specifici, un gateway comunica con endpoint noti. Tuttavia, definire il comportamento normale non è sempre semplice, soprattutto in presenza di aggiornamenti, cambiamenti operativi e rumore nei dati.

2.6 Machine learning applicato agli IDS

Il machine learning consente di apprendere relazioni tra feature di traffico e classi di appartenenza. Nei modelli supervisionati, il dataset contiene etichette che indicano se un campione è normale o malevolo, oppure a quale categoria di attacco appartiene. Nei modelli non supervisionati, invece, l'algoritmo cerca anomalie senza utilizzare etichette di attacco durante l'addestramento.

Random Forest, XGBoost e LightGBM sono modelli ensemble basati su alberi decisionali [5], [6], [7]. Sono adatti a dati tabulari e possono modellare relazioni non lineari. Logistic Regression è un modello lineare più semplice e interpretabile, utile come baseline e come riferimento per scenari embedded [20]. MLPClassifier rappresenta una rete neurale feed-forward implementata in scikit-learn [21]; offre maggiore flessibilità, ma richiede attenzione a convergenza e tuning. Isolation Forest è un metodo non supervisionato per anomaly detection, basato sull'idea che le anomalie siano più facili da isolare rispetto ai campioni normali [9].

2.7 Requisiti di sicurezza in un ambiente IoT

La sicurezza di un ambiente IoT deve essere analizzata rispetto ai requisiti classici di confidenzialità, integrità e disponibilità, ma questi requisiti assumono caratteristiche particolari quando sono applicati a dispositivi distribuiti e connessi. La confidenzialità riguarda la protezione dei dati raccolti dai sensori e trasmessi verso gateway o cloud. In molti scenari tali dati possono essere sensibili: informazioni sanitarie, posizione, abitudini domestiche, dati industriali o misure relative a processi produttivi.

L'integrità è altrettanto importante, perché un dato alterato può produrre decisioni errate. In una smart home, una misura falsificata può compromettere automazioni locali; in un contesto industriale, dati alterati possono influire su manutenzione predittiva o controllo di processo. La disponibilità, infine, è critica perché molti dispositivi IoT supportano servizi continui. Un attacco DoS verso un broker, un gateway o un dispositivo può impedire la raccolta di dati o l'attivazione di comandi.

Oltre alla triade confidenzialità-integrità-disponibilità, occorre considerare autenticazione, autorizzazione, tracciabilità e resilienza. L'autenticazione garantisce che solo dispositivi e utenti legittimi possano accedere al sistema. L'autorizzazione limita le operazioni consentite. La tracciabilità consente di ricostruire eventi e incidenti tramite log. La resilienza riguarda la capacità del sistema di continuare a funzionare, almeno parzialmente, anche in presenza di guasti o attacchi.

2.8 IDS in architetture edge e cloud

La collocazione dell'IDS influenza in modo significativo prestazioni, visibilità e costo. Un IDS collocato direttamente su un dispositivo può rilevare eventi locali con bassa latenza, ma è vincolato da memoria, CPU e consumo energetico. Un IDS collocato su gateway o edge node dispone di maggiore capacità computazionale e può osservare il traffico di più dispositivi. Un IDS collocato in cloud può eseguire analisi più complesse, ma dipende dalla trasmissione dei dati e può introdurre latenza o problemi di privacy.

In molti scenari è utile una soluzione ibrida. Il dispositivo può eseguire controlli semplici, il gateway può applicare modelli più complessi e il cloud può svolgere analisi storiche, correlazione degli eventi e aggiornamento periodico dei modelli. Questa impostazione consente di distribuire il carico computazionale e di adattare la risposta al livello di criticità.

La tesi adotta questa prospettiva: la pipeline Python rappresenta il livello sperimentale completo, adatto a training e valutazione; il dimostratore Wokwi rappresenta invece una versione ridotta e simulata dell'inferenza embedded. Il confronto tra questi due livelli permette di discutere il compromesso tra accuratezza e portabilità.

2.9 Difficoltà specifiche del machine learning per la sicurezza

L'applicazione del machine learning alla sicurezza presenta difficoltà che non emergono sempre in altri ambiti. La prima riguarda lo sbilanciamento delle classi: alcuni attacchi possono essere rari, mentre il traffico normale può essere molto più frequente. In questi casi, metriche come l'accuracy possono essere insufficienti. Per questo motivo la tesi considera Macro F1 e Weighted F1, oltre alle metriche più tradizionali.

La seconda difficoltà riguarda l'evoluzione degli attacchi. Un modello può imparare pattern presenti nel dataset, ma un attaccante può modificare comportamento, porte, frequenza dei pacchetti o payload per aggirare il rilevamento. La terza difficoltà riguarda la spiegabilità: un IDS che segnala un attacco deve fornire elementi utili per l'analisi, altrimenti rischia di produrre alert difficili da gestire.

La quarta difficoltà riguarda il deployment. Un modello addestrato in Python può funzionare bene su un computer, ma essere inadatto a un microcontrollore. Memoria, latenza, consumo energetico e aggiornabilità sono vincoli pratici che devono essere considerati fin dalla progettazione. Per questo motivo, nella tesi sono state incluse anche misure di dimensione e un esempio di modello embedded.

Capitolo 3 - Metodologia

3.1 Organizzazione del repository

Il repository è stato organizzato con l'obiettivo di rendere gli esperimenti riproducibili e verificabili. La struttura separa configurazione, dati, codice sorgente, script, risultati, documentazione di tesi e dimostrazione Wokwi. Questa separazione consente di evitare confusione tra dati grezzi, artefatti generati e codice.

La directory `config/` contiene il file `config.yaml`, che centralizza i percorsi e i parametri sperimentali. La directory `data/raw/` è destinata al dataset scaricato manualmente, mentre `data/processed/` può contenere eventuali dati trasformati. La directory `src/` contiene i moduli Python per caricamento dati, preprocessing, training, valutazione, visualizzazione, esperimenti e simulazione. La directory `results/` raccoglie metriche, grafici, modelli e log. La directory `thesis/` contiene la documentazione accademica in italiano.

Questa organizzazione evita l'uso di percorsi assoluti e consente di eseguire gli script dalla radice del repository. I file di grandi dimensioni e gli artefatti non adatti al versionamento, come dataset grezzi, modelli addestrati e ambiente virtuale, sono esclusi tramite `.gitignore`.

3.2 Dataset utilizzato

Il dataset utilizzato è TON_IoT/UNSW, nella componente di rete [1], [2], [13], [14]. Il file atteso dalla pipeline è `data/raw/train_test_network.csv`. La colonna `label` viene utilizzata come target per la classificazione binaria, mentre la colonna `type` viene utilizzata come target per la classificazione multiclasse.

La classificazione binaria distingue traffico normale e traffico di attacco. La classificazione multiclasse distingue invece più categorie di traffico, includendo classi normali e diverse tipologie di attacco. Il secondo compito è più complesso perché richiede di riconoscere non solo la presenza di una minaccia, ma anche la sua categoria.

Il dataset non è incluso nel repository. Questa scelta è necessaria sia per evitare di versionare file di grandi dimensioni sia per mantenere il progetto conforme a buone pratiche di gestione dei dati. Gli script verificano la presenza del file e, se il dataset manca, interrompono l'esecuzione con un messaggio chiaro senza produrre artefatti sperimentali.

3.3 Validazione dei dati

Prima del preprocessing, la pipeline controlla che il file atteso esista e che le colonne target richieste siano presenti. In caso di errore, il messaggio indica il percorso del dataset e fornisce istruzioni operative. Questo comportamento è importante perché evita errori ambigui, come eccezioni pandas o KeyError non comprensibili per chi esegue il progetto per la prima volta.

La validazione dei target è particolarmente rilevante perché dataset scaricati da fonti diverse possono avere nomi di file o colonne differenti. Nel progetto, la configurazione attesa è esplicita: label per il binario e type per il multiclasse. Se queste colonne non sono disponibili, la pipeline deve fermarsi e indicare le colonne presenti.

3.4 Preprocessing e prevenzione del data leakage

Il preprocessing include gestione dei valori mancanti, rimozione o gestione di colonne non adatte al training, codifica delle variabili categoriche e scaling delle variabili numeriche. La pipeline è stata progettata per evitare data leakage, cioè l'uso involontario di informazioni del test set durante l'addestramento.

In particolare, encoder e scaler vengono adattati solo sul training set. Successivamente, le trasformazioni apprese vengono applicate al test set. Questo schema rispecchia il comportamento atteso in un'applicazione reale: quando il modello viene usato su dati nuovi, non può conoscere in anticipo la distribuzione del test set.

Anche SMOTE viene applicato esclusivamente al training set. Applicare SMOTE al test set falserebbe la valutazione, perché modificherebbe artificialmente i dati su cui il modello deve essere testato. La pipeline mantiene quindi separati addestramento e valutazione.

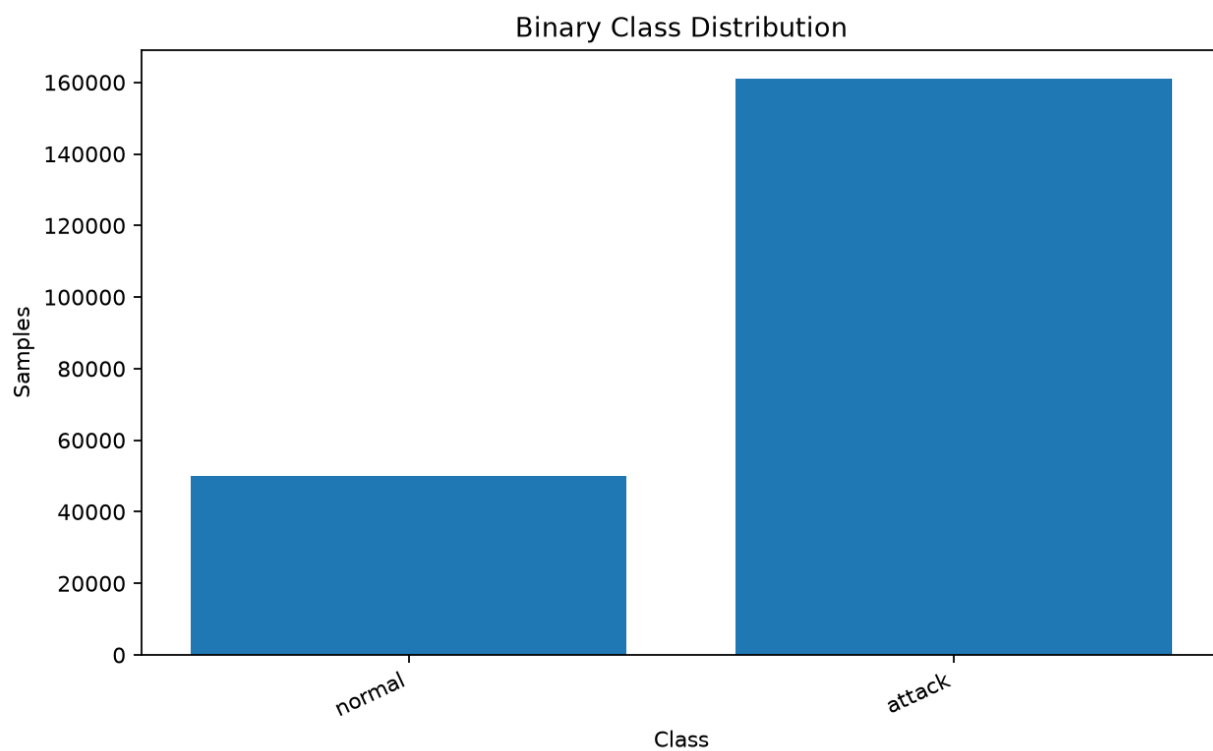


Figura 1 - Distribuzione delle classi binarie prima del preprocessing

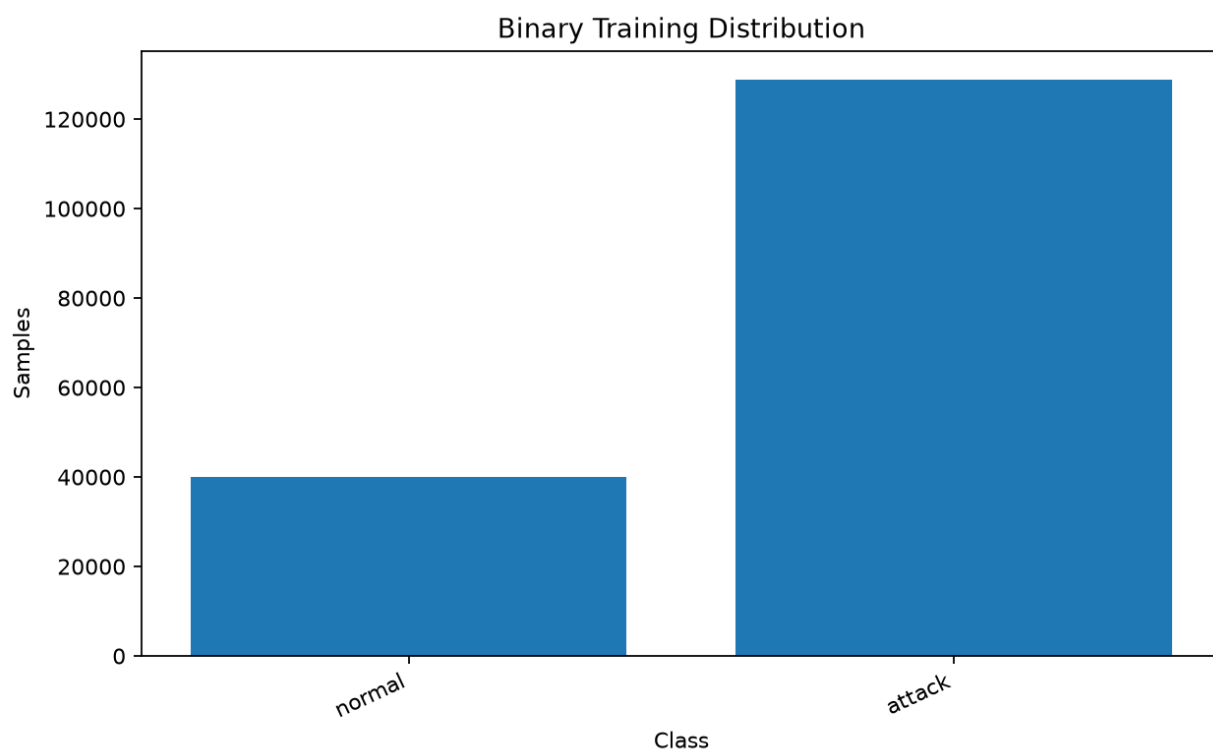


Figura 2 - Distribuzione delle classi binarie dopo il preprocessing

3.5 Split sperimentale

La suddivisione train/test è stratificata, con una quota di test pari al 20% e seed casuale fissato a 42. La stratificazione mantiene una distribuzione delle classi simile tra training set e test set, riducendo il rischio che una classe minoritaria sia rappresentata in modo insufficiente in una delle due partizioni.

Il seed fisso permette di ripetere l'esperimento ottenendo la stessa suddivisione e quindi metriche confrontabili. Questa scelta è essenziale in un progetto di tesi sperimentale, perché consente al relatore o ad altri studenti di verificare i risultati.

3.6 Modelli baseline

I modelli baseline sono Random Forest, LightGBM, XGBoost e Logistic Regression. Random Forest costruisce un insieme di alberi decisionali addestrati su sottoinsiemi dei dati e combina le predizioni. LightGBM e XGBoost sono modelli di gradient boosting, cioè costruiscono sequenzialmente alberi che correggono gli errori dei precedenti. Logistic Regression è un modello lineare che stima la probabilità di appartenenza a una classe.

La scelta di questi modelli consente di confrontare approcci diversi. I modelli ensemble sono generalmente più potenti su dati tabulari, mentre Logistic Regression è più interpretabile e leggera. Il confronto permette quindi di valutare non solo l'accuratezza, ma anche semplicità, tempo di inferenza e potenziale portabilità.

3.7 Esperimenti estesi

Gli esperimenti estesi includono SMOTE, tuning iperparametrico, MLPClassifier e Isolation Forest. SMOTE affronta lo sbilanciamento generando campioni sintetici della classe minoritaria nel training set [8]. Il tuning iperparametrico esplora configurazioni alternative dei modelli ensemble per verificare eventuali margini di miglioramento. MLPClassifier introduce un modello neurale feed-forward. Isolation Forest valuta un approccio di anomaly detection non supervisionato [9].

Il dataset contiene 211.045 record: con lo split stratificato 80/20, le baseline usano 168.836 record di training e 42.209 record di test. Per rendere sostenibili e confrontabili gli esperimenti estesi, SMOTE, tuning e MLPClassifier usano campioni stratificati di 30.000 record di training sia nel caso binario sia nel multiclasse; la valutazione resta sempre effettuata sull'intero test set separato di 42.209 record. Per il confronto con

SMOTE, anche il training set bilanciato viene ricampionato a 30.000 record dopo il sovracampionamento. Isolation Forest utilizza soltanto i 7.108 record normali presenti nel campione binario, coerentemente con l'addestramento one-class. Il tuning valuta 6 configurazioni casuali con 3 fold di cross-validation. Le numerosità effettive sono salvate nei CSV nelle colonne training_samples e test_samples.

Sono state inoltre introdotte analisi complementari: interpretabilità SHAP [10], misure di latenza e dimensione degli artefatti, compressione dei modelli e generazione di un modello compatto esportabile in C++ per simulazione Wokwi.

3.8 Metriche di valutazione

Le metriche principali sono accuracy, precision, recall, F1-score, Macro F1 e Weighted F1. L'accuracy misura la quota complessiva di predizioni corrette, ma può essere fuorviante in presenza di classi sbilanciate. Precision e recall permettono di valutare falsi positivi e falsi negativi. L'F1-score combina precision e recall. Macro F1 calcola la media non pesata sulle classi ed è utile per evidenziare difficoltà sulle classi minoritarie. Weighted F1 pesa le classi in base alla loro numerosità.

Per ogni modello vengono inoltre salvate matrice di confusione, report di classificazione, tempo di training e tempo di inferenza. Le matrici di confusione consentono di analizzare quali classi vengono confuse tra loro. I tempi di training e inferenza sono importanti perché un IDS deve poter operare entro vincoli pratici, soprattutto in scenari edge o gateway.

3.9 Protocollo di esecuzione

Il protocollo sperimentale è stato definito per essere ripetibile. Dopo la preparazione dell'ambiente Python e l'installazione delle dipendenze, il dataset deve essere collocato nel percorso previsto. Successivamente, gli esperimenti possono essere eseguiti tramite gli script presenti in src/experiments/. Lo script run_binary.py esegue la baseline binaria, run_multiclass.py la baseline multiclasse e run_extended.py gli esperimenti con SMOTE, tuning, MLPClassifier e Isolation Forest.

Il nome run_all.py indica l'orchestrazione del nucleo classificativo: lo script richiama, nell'ordine, run_binary.py, run_multiclass.py, run_extended.py e generate_result_summary.py. Non richiama run_interpretability.py, run_deployment_analysis.py, export_embedded_model.py né la simulazione

MQTT/Wokwi. Queste analisi sono eseguite separatamente perché presentano dipendenze, costi computazionali o ambienti di esecuzione differenti; i comandi dedicati sono documentati nel README.

Ogni runner segue lo stesso schema logico: caricamento della configurazione, validazione del dataset, preprocessing, addestramento, valutazione, salvataggio delle metriche e generazione dei grafici. Questo schema riduce la duplicazione e consente di individuare rapidamente eventuali errori.

La pipeline è stata controllata anche rispetto al caso in cui il dataset non sia presente. In tale scenario, gli script terminano con un messaggio esplicito e non producono CSV o PNG di risultato. Questa scelta è importante dal punto di vista metodologico: in una tesi sperimentale, i risultati devono derivare da esecuzioni reali.

3.10 Gestione degli artefatti

Gli output sono organizzati nella directory results/. Le metriche tabulari sono salvate in results/metrics/, i grafici in results/plots/, gli artefatti dei modelli in results/models/ e i log in results/logs/. Il repository consente di versionare metriche e grafici generati da esperimenti reali, mentre esclude dataset grezzi, modelli addestrati e log.

Questa distinzione è utile perché le metriche e i grafici sono documentazione sperimentale, mentre dataset e modelli possono essere grandi, sensibili o rigenerabili. I modelli addestrati non sono necessari per comprendere il lavoro se il codice e il dataset sono disponibili; possono essere rigenerati tramite gli script.

3.11 Simulazione IoT e dimostrazione embedded

Accanto alla pipeline su dataset reale, il progetto include una componente di simulazione IoT. La simulazione Docker/MQTT [11] mostra come dispositivi simulati possano pubblicare messaggi, come un collector possa raccogliarli e come un attacco simulato possa produrre pattern anomali. Questa parte non sostituisce TON_IoT, ma supporta la discussione architetturale.

Il dimostratore Wokwi [12] rappresenta un passaggio ulteriore: un modello compatto viene esportato in C++ e simulato su ESP32. L'obiettivo non è ottenere le stesse prestazioni di LightGBM, ma mostrare come un classificatore leggero possa essere incorporato in un ambiente embedded, secondo la prospettiva TinyML [19]. La presenza del dimostratore rende più concreta la discussione sui vincoli IoT.

Capitolo 4 - Risultati sperimentali

4.1 Premessa sui risultati

Tutti i valori riportati in questo capitolo derivano dai file CSV generati dalla pipeline Python dopo l'esecuzione degli esperimenti sul dataset locale. I grafici inclusi nel documento sono presenti nella directory results/plots/ e sono stati prodotti dagli script del repository.

I risultati vanno interpretati tenendo conto del contesto sperimentale. Il dataset è statico e già etichettato; le prestazioni ottenute indicano la capacità dei modelli di classificare quel dataset secondo la suddivisione adottata, non garantiscono automaticamente prestazioni identiche in reti reali con traffico variabile, dispositivi diversi o attacchi non osservati in addestramento.

4.2 Risultati baseline nella classificazione binaria

Tabella 1 - Prestazioni dei modelli baseline nella classificazione binaria

Modello	Accuracy	F1-score	Macro F1	Training (s)	Inference (s)
Random Forest	0.9988	0.9992	0.9983	24.43	0.283
LightGBM	0.9992	0.9995	0.9989	3.50	0.215
XGBoost	0.9988	0.9992	0.9983	5.46	0.066
Logistic Regression	0.9576	0.9722	0.9414	5.72	0.008

Nella classificazione binaria, LightGBM ottiene la migliore accuracy e il migliore F1-score tra i modelli baseline. Random Forest e XGBoost mostrano risultati molto vicini, con differenze minime sul piano predittivo. Logistic Regression ottiene valori inferiori, ma conserva un tempo di inferenza particolarmente ridotto.

La matrice di confusione di LightGBM mostra 9987 campioni normali classificati correttamente, 13 falsi positivi, 32188 attacchi classificati correttamente e 21 falsi negativi. Questo risultato indica un numero molto basso di errori rispetto alla dimensione del test set. In un contesto IDS, i falsi negativi sono particolarmente critici perché rappresentano attacchi non rilevati; i falsi positivi, invece, possono generare alert inutili e affaticamento operativo.

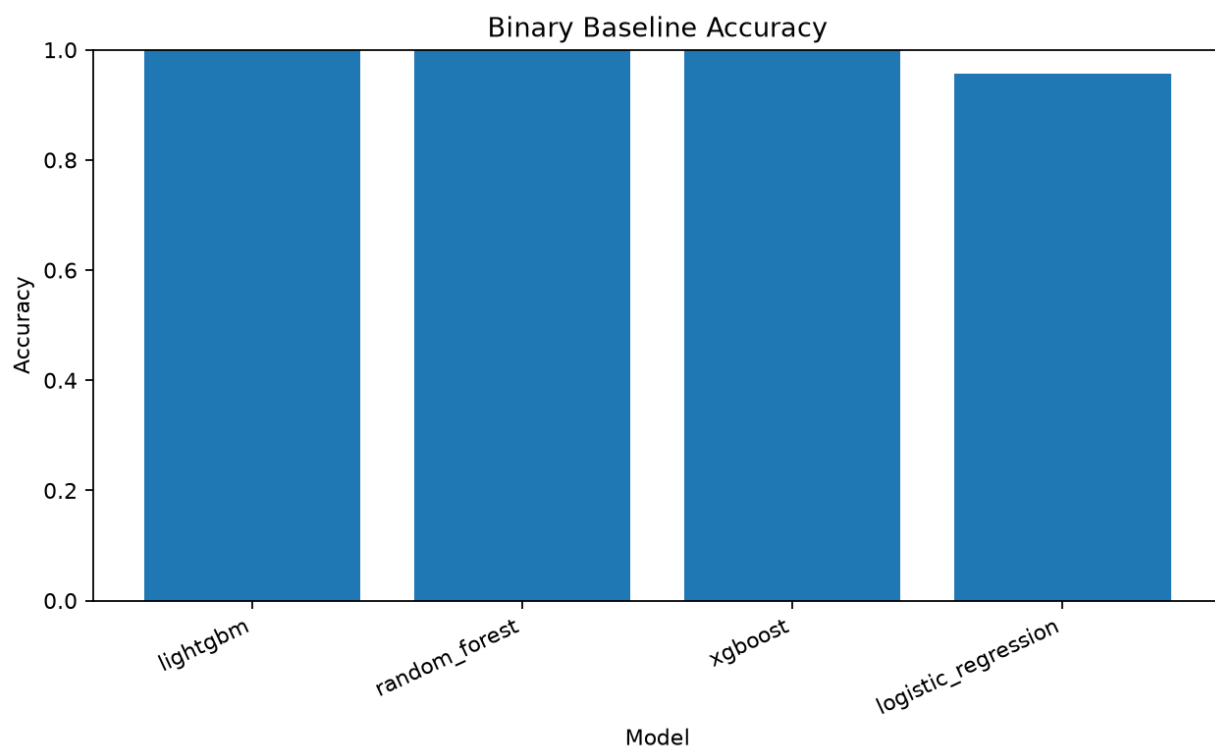


Figura 3 - Confronto dell'accuracy dei modelli baseline nella classificazione binaria

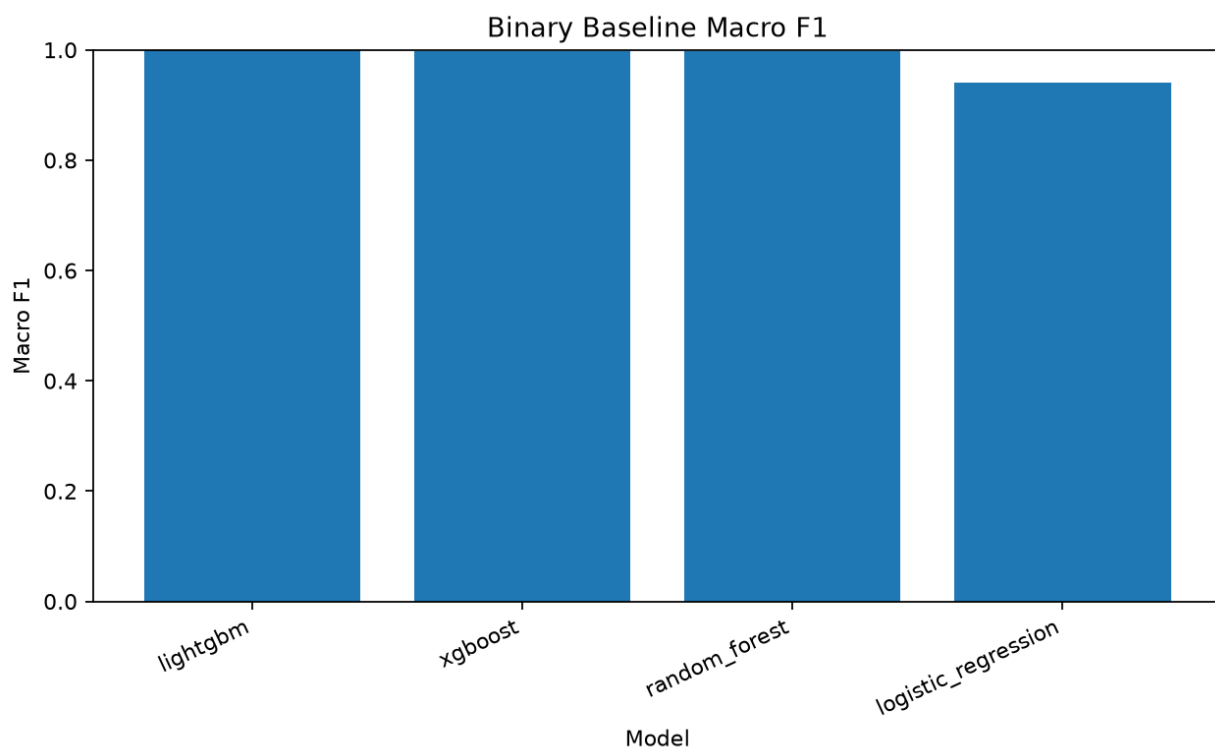


Figura 4 - Confronto del Macro F1 dei modelli baseline nella classificazione binaria

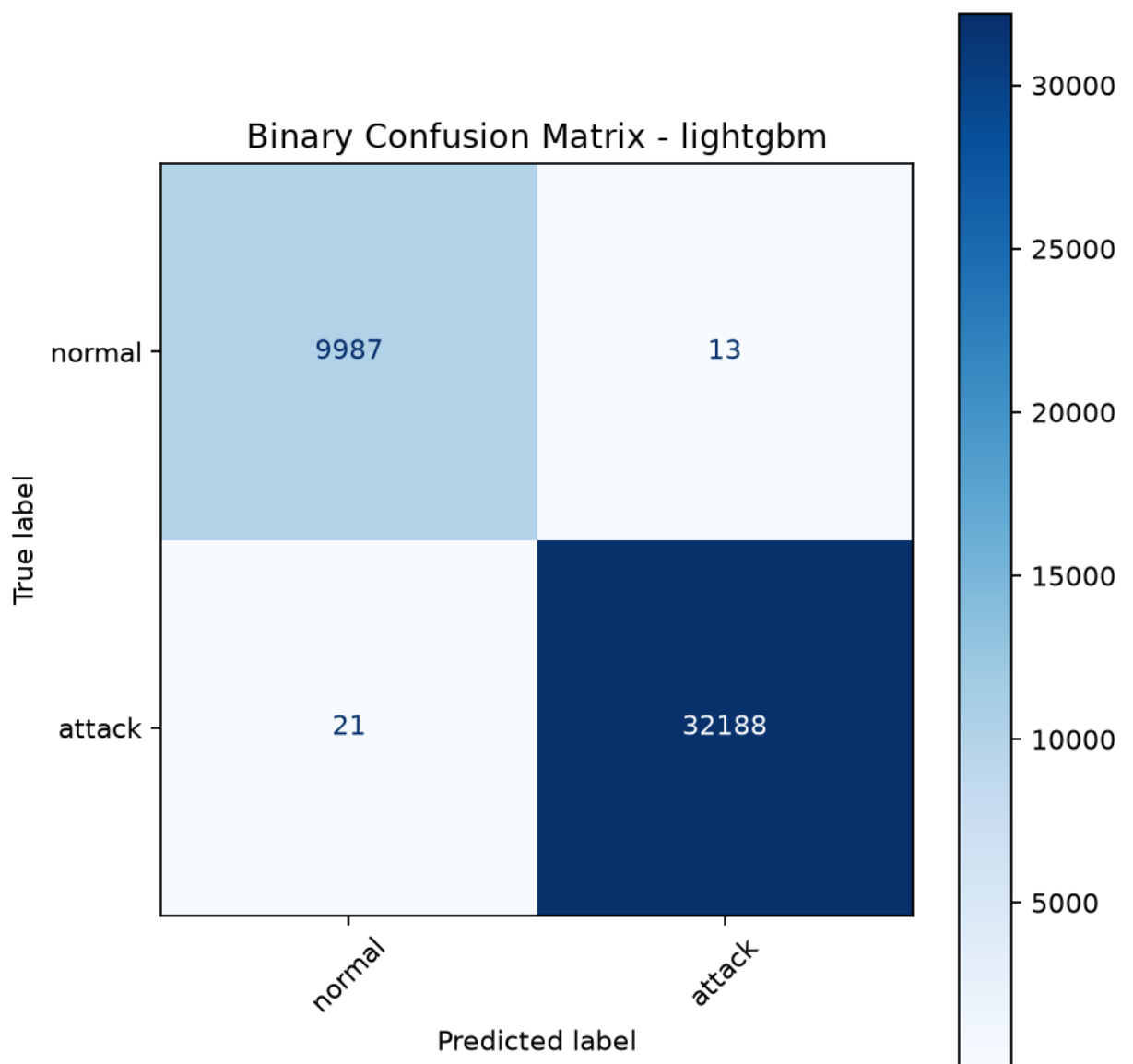


Figura 5 - Matrice di confusione LightGBM nella classificazione binaria

4.3 Risultati baseline nella classificazione multiclasse

Tabella 2 - Prestazioni dei modelli baseline nella classificazione multiclasse

Modello	Accuracy	Weighted F1	Macro F1	Training (s)	Inference (s)
Random Forest	0.9883	0.9884	0.9654	23.63	0.539
LightGBM	0.9897	0.9898	0.9678	26.82	2.422
XGBoost	0.9891	0.9893	0.9655	58.96	0.557
Logistic Regression	0.8263	0.8330	0.7686	211.00	0.022

Nel caso multiclasse, LightGBM ottiene la migliore accuracy baseline e il valore più elevato di Macro F1. XGBoost e Random Forest rimangono competitivi, mentre Logistic Regression presenta limiti più evidenti. Il calo di Logistic Regression è coerente con la maggiore complessità del problema multiclasse: distinguere categorie di attacco differenti richiede confini decisionali più articolati rispetto alla semplice separazione normale/attacco.

Il Macro F1 è particolarmente importante nella valutazione multiclasse, perché assegna lo stesso peso a ciascuna classe. Un modello potrebbe ottenere accuracy elevata classificando bene le classi più numerose, ma avere prestazioni scarse sulle classi minoritarie. Il fatto che LightGBM mantenga anche il Macro F1 più alto suggerisce una capacità più equilibrata di distinguere le diverse categorie.

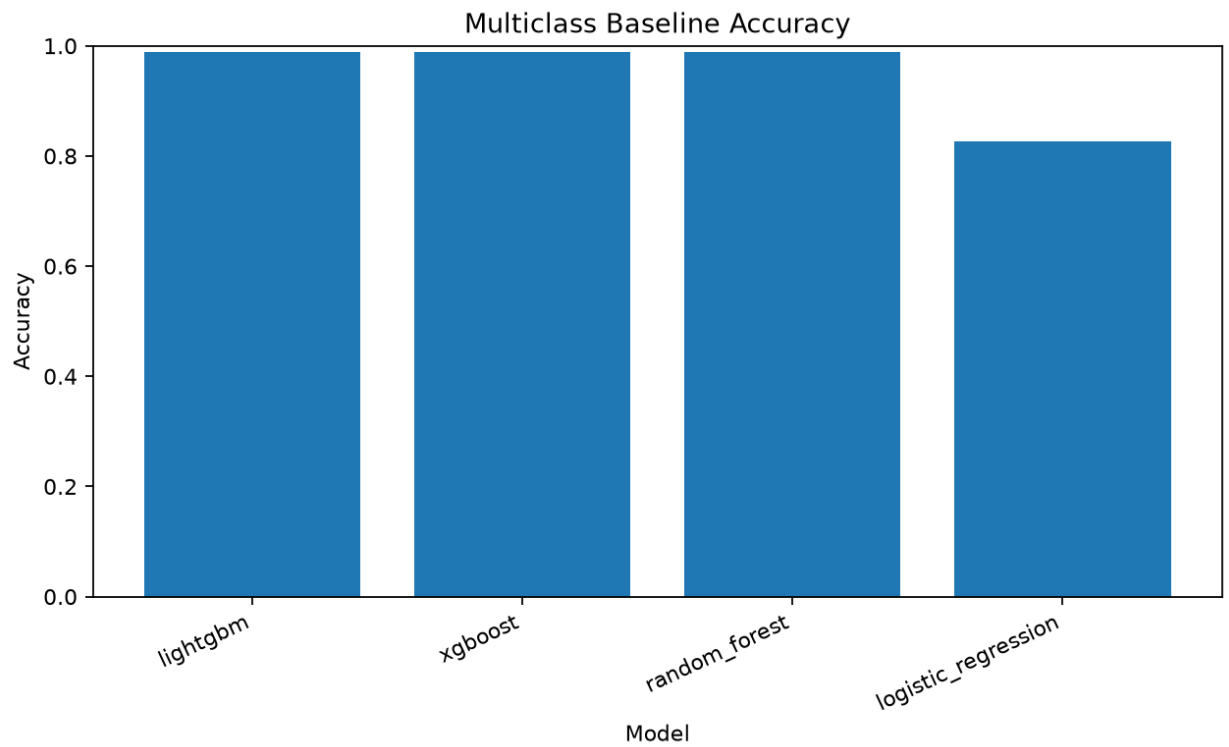


Figura 6 - Confronto dell'accuracy dei modelli baseline nella classificazione multiclasse

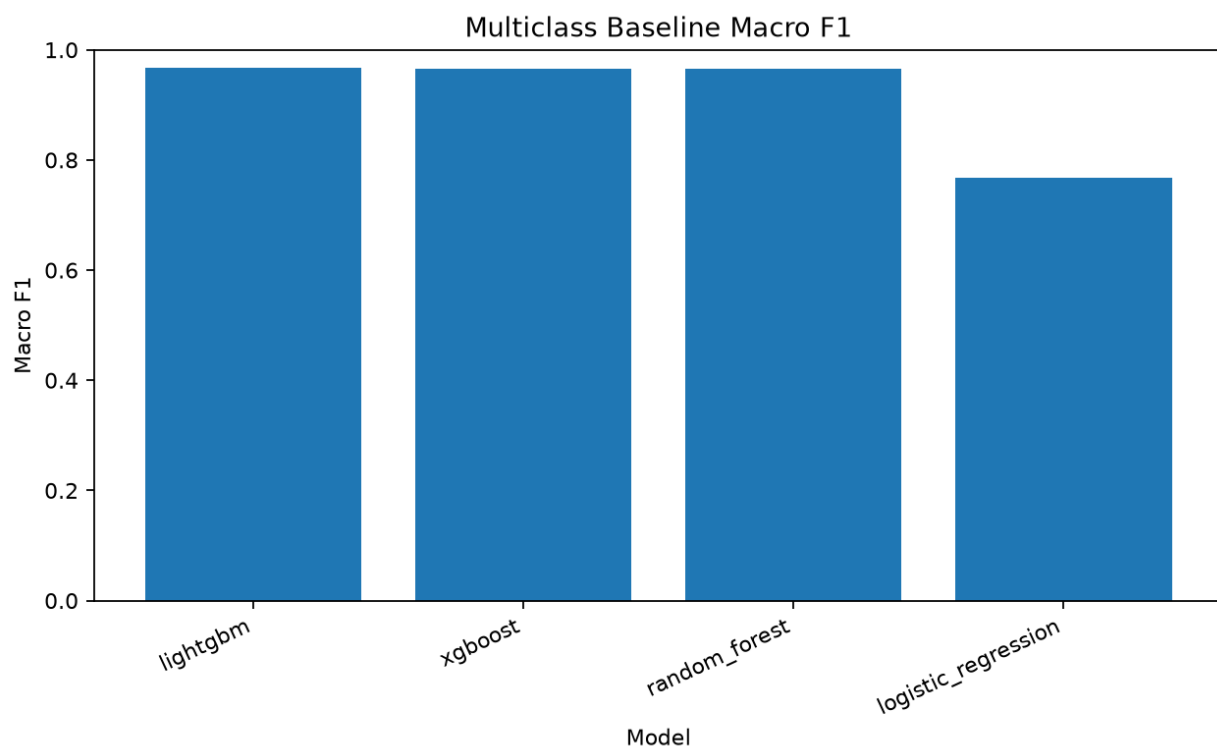


Figura 7 - Confronto del Macro F1 dei modelli baseline nella classificazione multiclasse

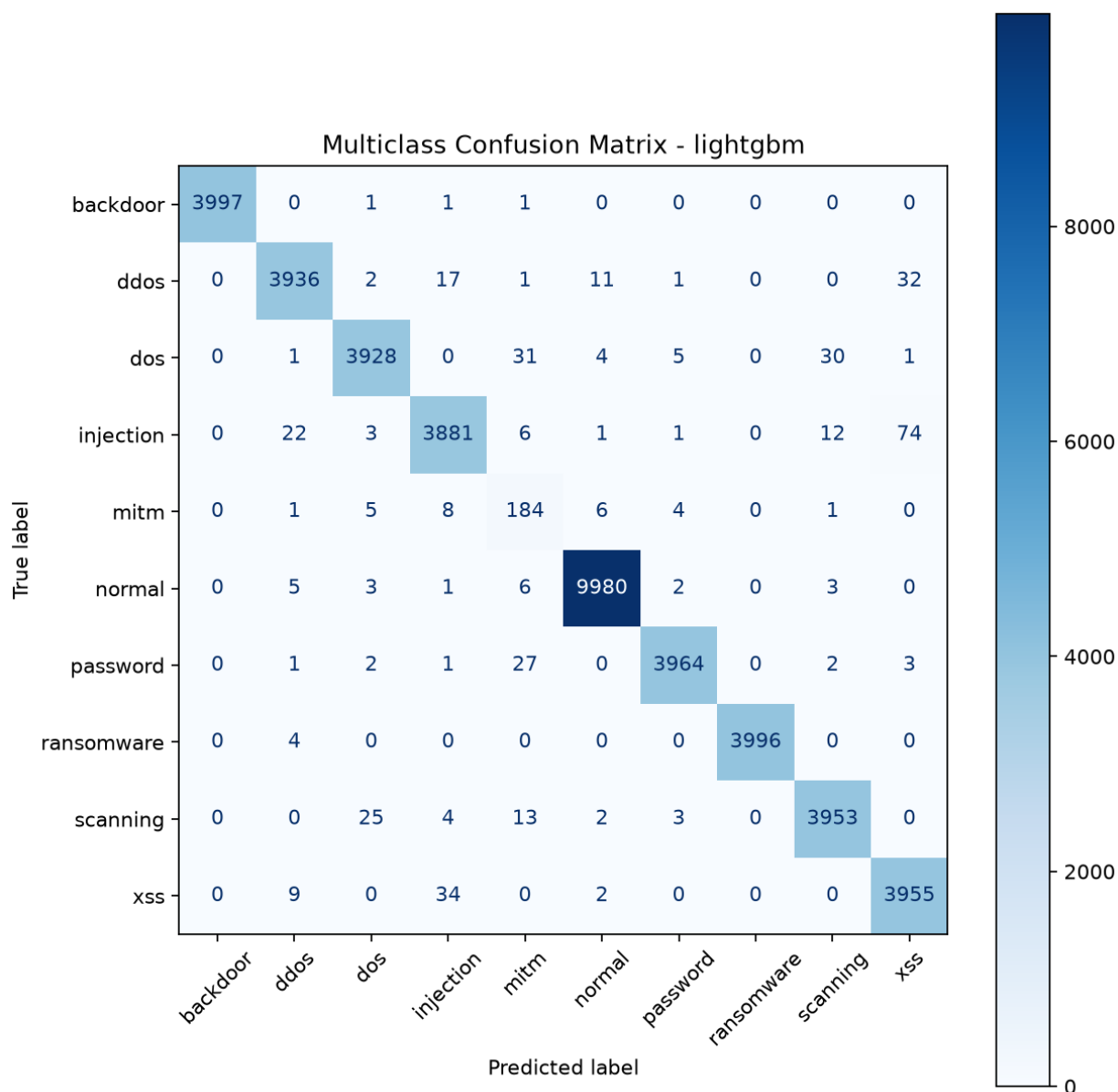


Figura 8 - Matrice di confusione LightGBM nella classificazione multiclasse

4.4 Effetto del bilanciamento tramite SMOTE

SMOTE è stato applicato solo al training set, in modo da evitare data leakage. L'obiettivo era verificare se la generazione di campioni sintetici per classi meno rappresentate potesse migliorare le prestazioni, in particolare il Macro F1.

Nel caso binario, Random Forest senza SMOTE ottiene Macro F1 pari a 0.9974, mentre con SMOTE ottiene 0.9969. Per Logistic Regression il Macro F1 passa da 0.9408 a 0.9410. Nel caso multiclasse, LightGBM passa da 0.9524 senza SMOTE a 0.9528 con SMOTE.

I risultati indicano che SMOTE non produce un miglioramento sistematico. In alcuni casi l'effetto è marginalmente positivo, in altri è marginalmente negativo. Questo è un risultato utile: il bilanciamento non deve essere applicato come automatismo, ma valutato sperimentalmente. Se le classi sono già separabili o se il modello gestisce bene lo sbilanciamento, l'aggiunta di campioni sintetici può non portare benefici significativi.

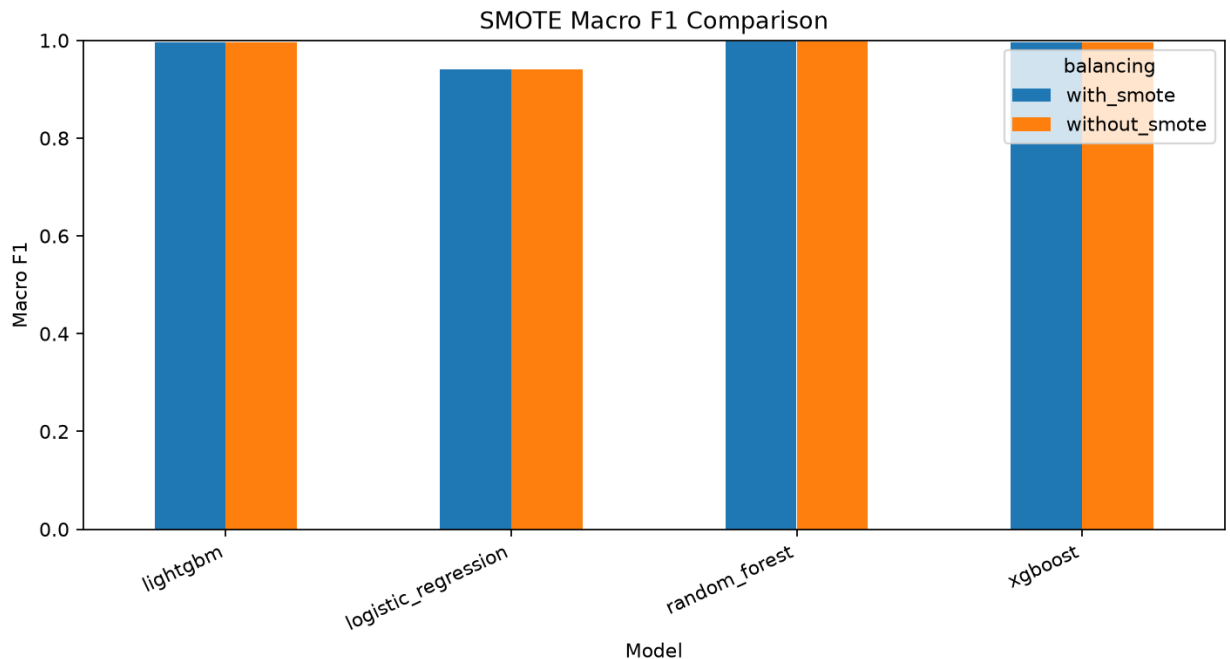


Figura 9 - Confronto Macro F1 con e senza SMOTE

4.5 Tuning iperparametrico

Il tuning iperparametrico è stato eseguito con RandomizedSearchCV, utilizzando 6 iterazioni e 3 fold di cross-validation. Per contenere il costo computazionale è stato usato un campione stratificato del training set. Questa scelta rende l'esperimento sostenibile su ambiente locale, ma impone cautela nell'interpretazione: la baseline principale utilizza l'intero training set, mentre il tuning opera su un sottoinsieme.

Nel caso binario, LightGBM ottiene Macro F1 pari a 0.9979, Random Forest 0.9974 e XGBoost 0.9972. Nel caso multiclasse, Random Forest ottiene Macro F1 pari a 0.9528, LightGBM 0.9523 e XGBoost 0.9482. I valori sono elevati, ma non mostrano un miglioramento netto rispetto alla baseline completa.

Questo risultato suggerisce due considerazioni. La prima è che le configurazioni baseline sono già molto competitive sul dataset considerato. La seconda è che un tuning più esteso, con più iterazioni, più fold o strumenti come Optuna, potrebbe essere utile

per una prosecuzione del lavoro, ma richiederebbe risorse computazionali superiori e tempi di esecuzione più lunghi.

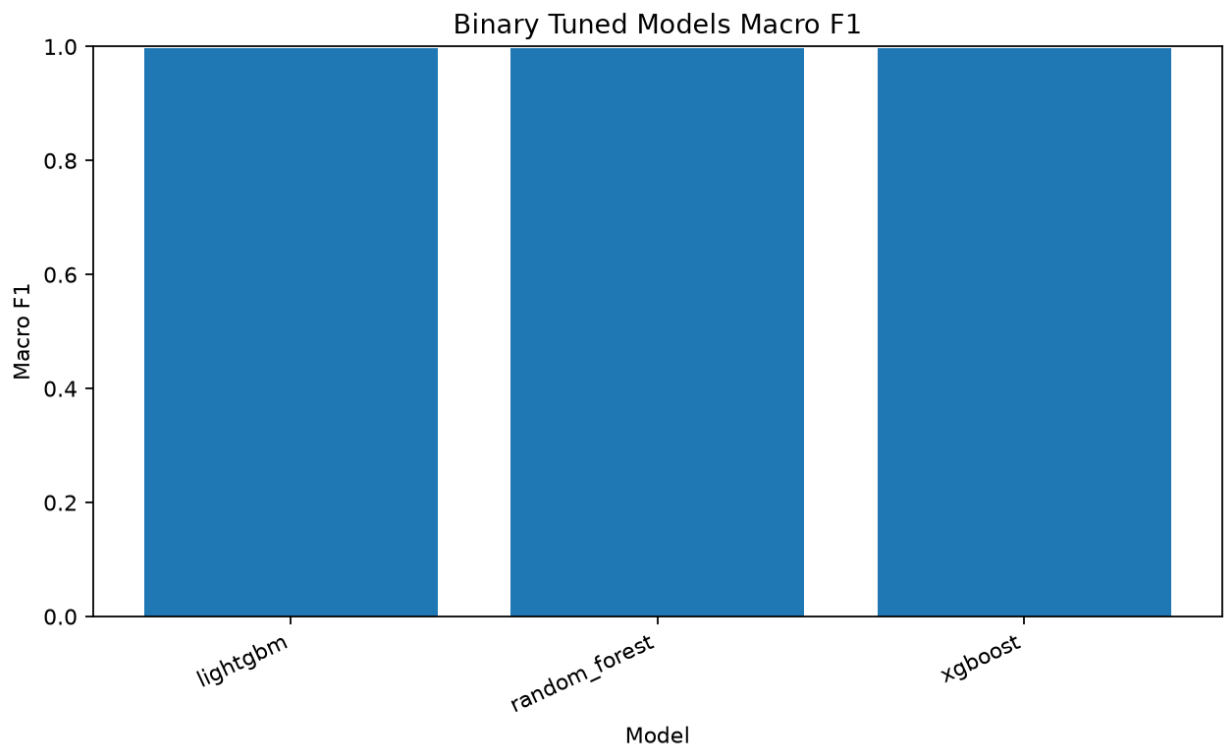


Figura 10 - Confronto Macro F1 dei modelli sottoposti a tuning binario

4.6 Modelli aggiuntivi: MLPClassifier e Isolation Forest

MLPClassifier è stato introdotto come modello neurale feed-forward. Nella classificazione binaria ottiene Macro F1 pari a 0.9899, mentre nella classificazione multiclasse ottiene Macro F1 pari a 0.8851. Il risultato binario è buono, ma rimane inferiore ai modelli ensemble migliori. Nel caso multiclasse, il modello presenta maggiori difficoltà e genera un warning di convergenza, indicando che l'ottimizzazione potrebbe richiedere più iterazioni, una diversa architettura o una migliore configurazione degli iperparametri.

Isolation Forest è stato valutato come approccio non supervisionato per anomaly detection binaria. Il modello ottiene Macro F1 pari a 0.3625, sensibilmente inferiore ai modelli supervisionati. Questo risultato è coerente con la natura del metodo: Isolation Forest non sfrutta direttamente le etichette di attacco durante l'addestramento e quindi opera con meno informazione rispetto ai classificatori supervisionati.

La presenza di risultati inferiori non è un errore sperimentale, ma un elemento importante della discussione. Mostra che non tutti i modelli sono adatti allo stesso modo

al problema e che il contesto supervisionato del dataset favorisce modelli addestrati con etichette.

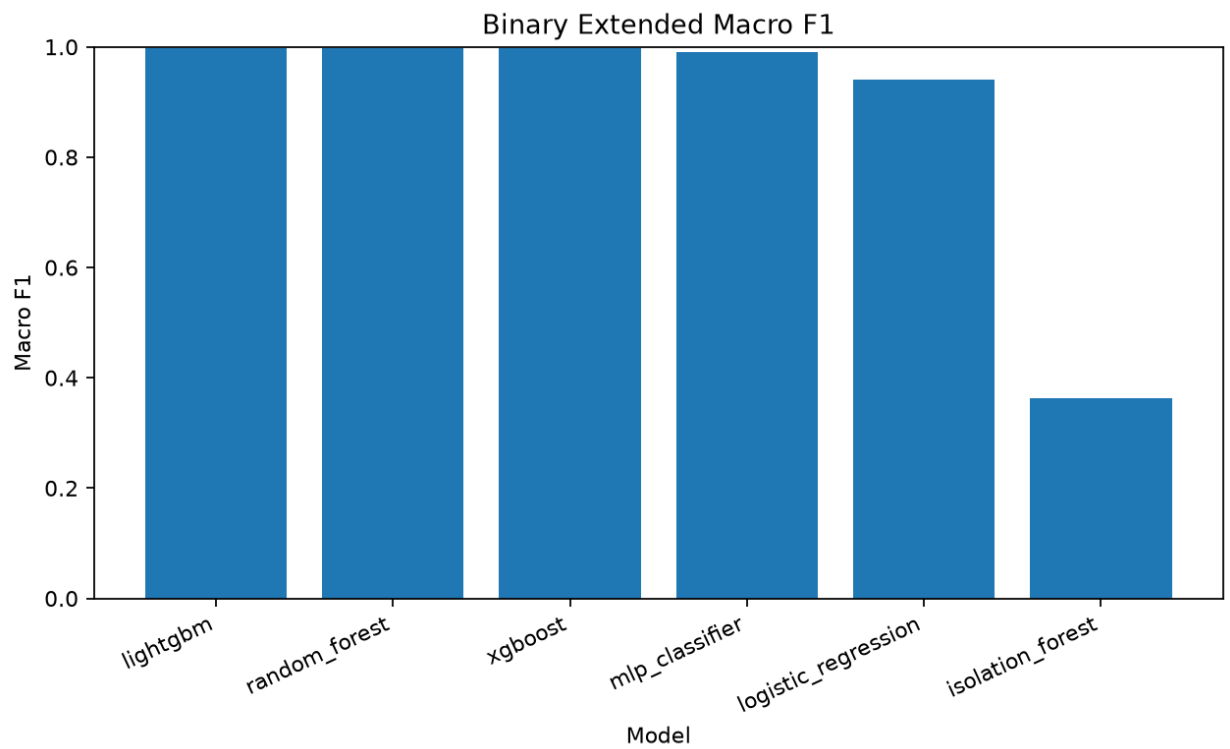


Figura 11 - Confronto Macro F1 degli esperimenti estesi binari

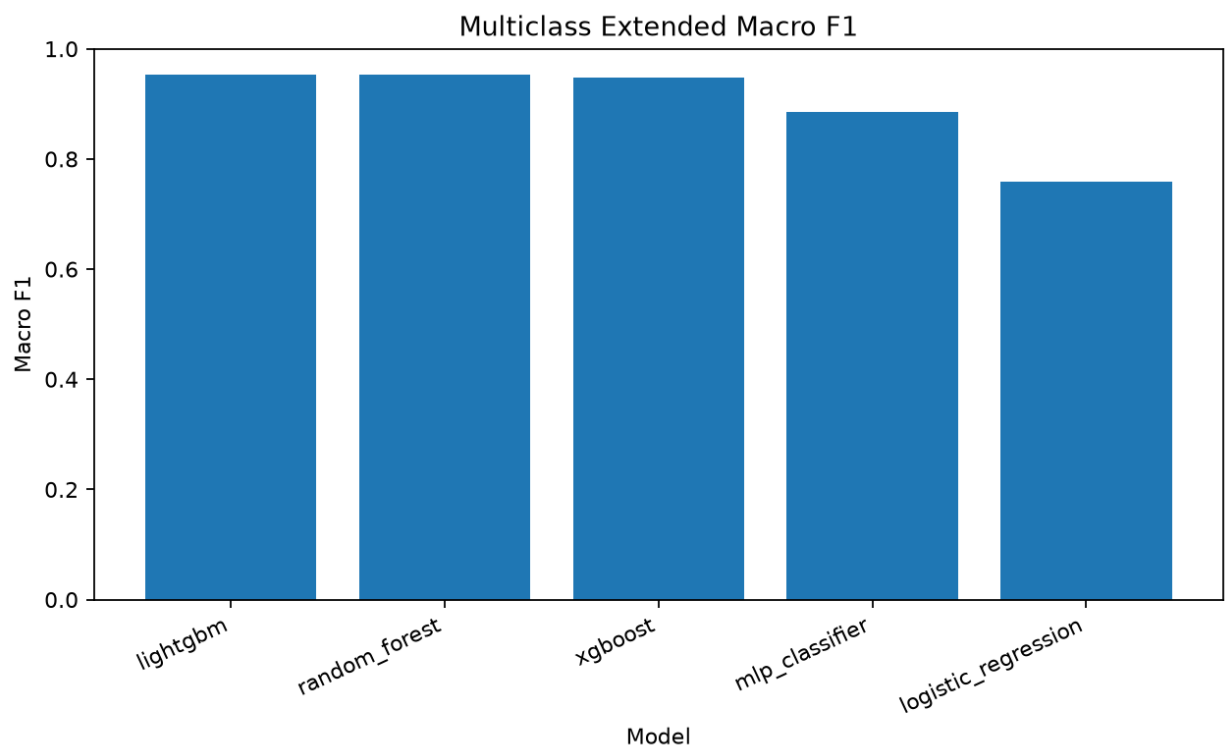


Figura 12 - Confronto Macro F1 degli esperimenti estesi multiclasse

4.7 Interpretabilità dei modelli

L'interpretabilità è un requisito importante per un IDS, soprattutto quando le decisioni del modello generano alert o interventi operativi. Un modello molto accurato ma completamente opaco può essere difficile da accettare in contesti reali, perché gli amministratori devono comprendere almeno quali feature contribuiscono maggiormente alle decisioni.

Nel progetto è stata eseguita un'analisi SHAP globale sul modello LightGBM, selezionato perché migliore baseline in base al Macro F1. L'importanza è calcolata come media del valore SHAP assoluto su 1.000 campioni del test set [10]. Nel caso binario, le feature principali sono `dst_port` (2.3037), `proto_tcp` (2.0220), `proto_udp` (1.8811), `conn_state_REJ` (1.4085) e `src_pkts` (1.2056). Nel caso multiclasse emergono `dst_port` (0.9983), `src_port` (0.8610), `src_ip_bytes` (0.6687), `duration` (0.6638) e `conn_state_REJ` (0.4237). Porte, protocolli, stato e volume del flusso sono caratteristiche coerenti con il dominio del traffico di rete.

L'analisi non deve essere interpretata come spiegazione causale completa: il valore SHAP quantifica quanto una feature contribuisce alle predizioni rispetto al valore di riferimento del modello, ma non dimostra da solo perché un evento sia malevolo. L'aggregazione assoluta adottata fornisce una lettura globale confrontabile tra feature; le spiegazioni locali di singoli alert rimangono fuori dal perimetro sperimentale dichiarato.

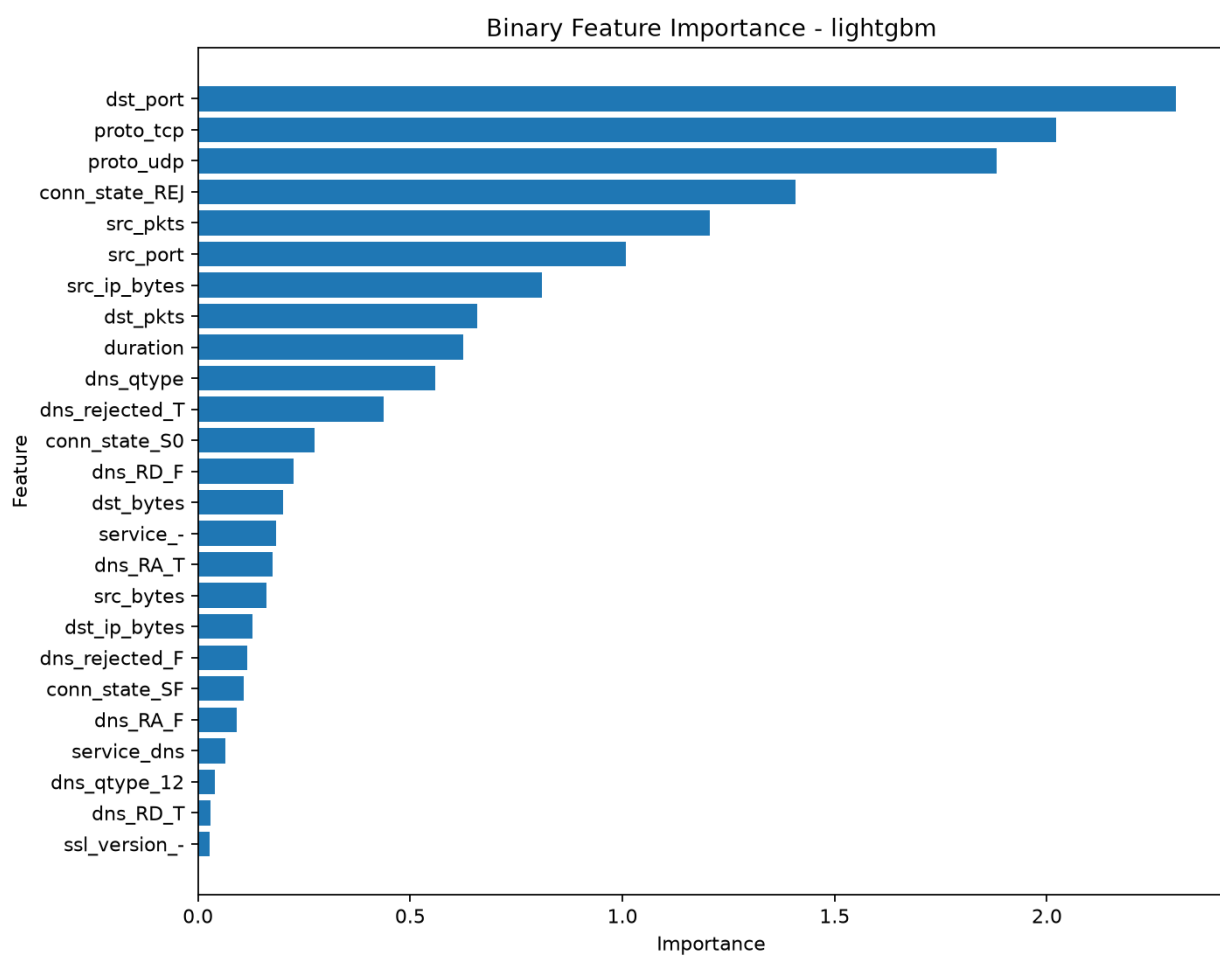


Figura 13 - Importanza delle feature nel modello LightGBM binario

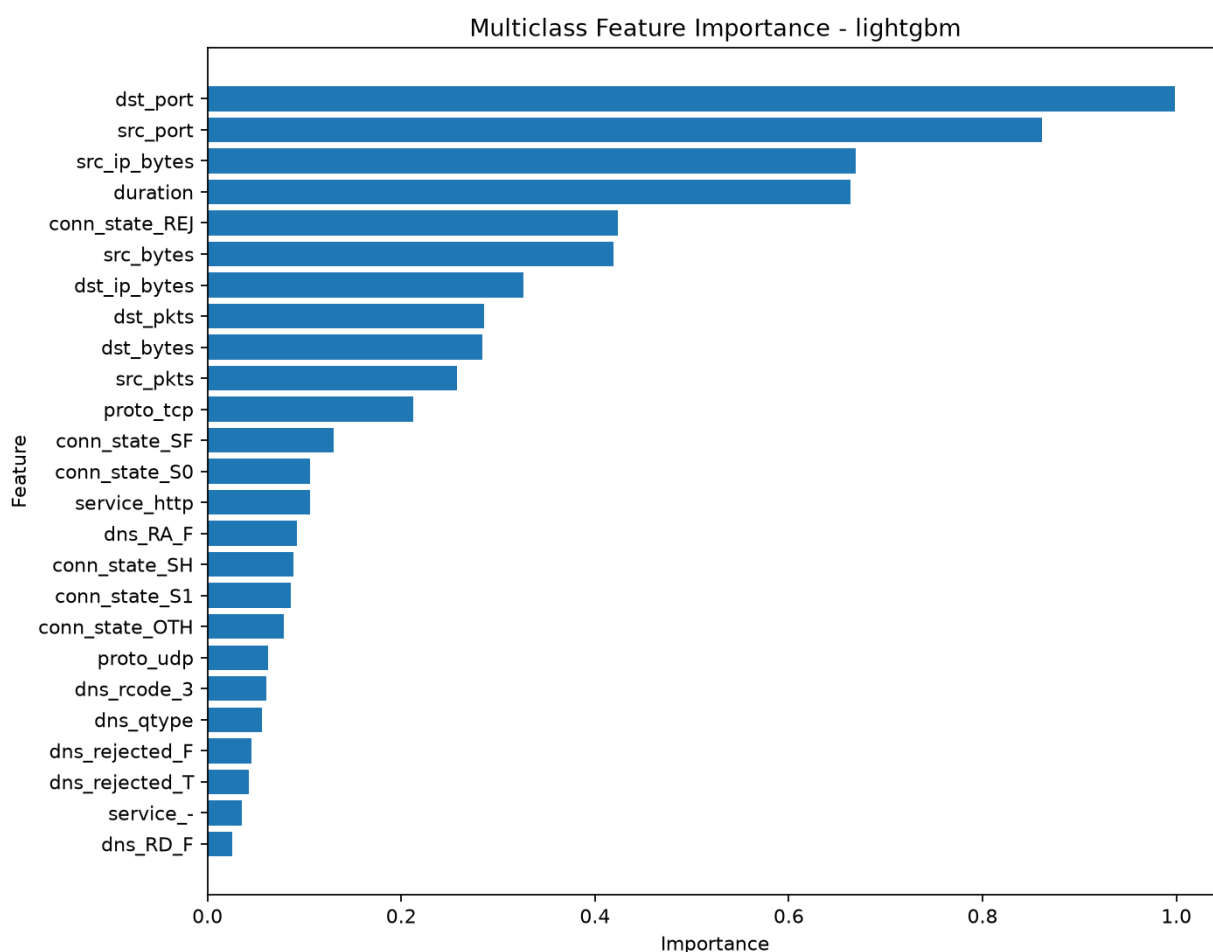


Figura 14 - Importanza delle feature nel modello LightGBM multiclasse

4.8 Latenza, dimensione e compressione

Oltre alle metriche predittive, sono state considerate misure legate al deployment. In un sistema IDS reale, il modello deve elaborare traffico in tempi compatibili con i requisiti operativi. Un modello accurato ma troppo lento o troppo grande potrebbe essere inadatto a dispositivi edge o gateway con risorse limitate.

L'analisi di deployment tramite Joblib mostra una riduzione della dimensione degli artefatti LightGBM: da 2.09 MB a 0.85 MB nel caso binario e da 17.20 MB a 7.34 MB nel caso multiclasse. Questi valori riguardano la compressione dell'artefatto Python e non equivalgono a una quantizzazione per microcontrollori. Sono comunque utili per discutere il rapporto tra prestazioni, dimensione e portabilità.

Le misure di inferenza mostrano che Logistic Regression è molto veloce, ma meno accurata. LightGBM offre invece il miglior compromesso complessivo nella baseline, con accuracy elevata e tempi accettabili. In uno scenario reale, la scelta del modello dipenderebbe dal punto di deployment: cloud, gateway o dispositivo embedded.

4.9 Dimostratore embedded con Wokwi

È stato realizzato un dimostratore Wokwi su ESP32 per simulare l'esecuzione embedded di un modello IDS compatto. L'exporter addestra una Logistic Regression binaria su dieci feature numeriche, normalizza le variabili, quantizza coefficienti e intercetta in int16 e genera un header C++ utilizzabile nello sketch Arduino.

Per distinguere l'effetto della quantizzazione dalle prestazioni del classificatore, l'exporter valuta sul medesimo test set di 42.209 record sia la pipeline Python originale in virgola mobile sia un'emulazione Python della formula quantizzata eseguita dal firmware. Entrambe le varianti ottengono accuracy 0.8180, F1-score 0.8888 e Macro F1 0.6933. L'accordo tra le predizioni è pari al 100%; le differenze di accuracy e Macro F1 sono entrambe 0.0000. Alla scala di quantizzazione adottata, quindi, l'arrotondamento dei coefficienti non altera le classi predette su questo test set. Il confronto non misura invece consumo energetico o latenza su hardware fisico.

Il progetto Wokwi è disponibile al seguente indirizzo:

<https://wokwi.com/projects/472799587810026497>

Il dimostratore contiene tre file principali: `sketch.ino`, `embedded_model.h` e `diagram.json`. Lo sketch inizializza la seriale, carica campioni di test, calcola la probabilità di attacco e stampa per ogni campione etichetta attesa, etichetta predetta e probabilità. Il file `diagram.json` collega esplicitamente `esp:TX` e `esp:RX` al monitor seriale virtuale, permettendo di osservare l'output durante la simulazione.

Il dimostratore non usa LightGBM. Questa scelta è intenzionale: LightGBM è il modello più efficace nella baseline, ma è più complesso da incorporare direttamente in uno sketch minimale per microcontrollore. Logistic Regression consente invece di rappresentare l'inferenza come prodotto scalare tra feature normalizzate e coefficienti quantizzati, seguito da una funzione sigmoide. Il risultato è meno accurato, ma più adatto a mostrare il passaggio concettuale da pipeline Python a inferenza embedded.

L'output seriale del simulatore mostra sia predizioni corrette sia errori. Questo aspetto è importante perché conferma che il dimostratore non presenta una sequenza artificiale perfetta, ma riflette le prestazioni inferiori del modello compatto rispetto alla baseline ensemble.

4.10 Sintesi degli artefatti prodotti

Gli esperimenti hanno prodotto file CSV, grafici PNG e documentazione tecnica. I principali file di metriche sono `binary_baseline_metrics.csv`, `multiclass_baseline_metrics.csv`, `binary_extended_metrics.csv`, `multiclass_extended_metrics.csv`, `smote_comparison.csv`, `hyperparameter_tuning_results.csv`, `latency_summary.csv`, `deployment_analysis.csv` ed `embedded_logistic_regression_metrics.csv`.

I grafici principali includono confronti per accuracy, F1-score, Macro F1, matrici di confusione, distribuzioni delle classi e feature importance. Tali figure sono incluse nel documento per rendere immediata la lettura dei risultati e per collegare le metriche numeriche alla loro rappresentazione visuale.

La presenza di artefatti generati automaticamente rafforza la riproducibilità del lavoro. Un lettore può confrontare le tabelle riportate nel testo con i CSV nella directory `results/metrics/` e verificare che le immagini inserite provengano dai file presenti in `results/plots/`.

4.11 Lettura critica dei risultati

Le prestazioni molto elevate dei modelli ensemble devono essere interpretate con attenzione. Da un lato indicano che le feature del dataset sono informative e che i modelli riescono a distinguere efficacemente traffico normale e malevolo. Dall'altro lato, valori molto alti su dataset statici possono dipendere anche dalla struttura del dataset, dalla separabilità delle classi e dalla somiglianza tra training e test set.

Per questo motivo, il lavoro non afferma che un modello addestrato su TON_IoT sia automaticamente sufficiente per proteggere una rete reale. Il risultato sperimentale dimostra la validità della pipeline e la capacità dei modelli di apprendere dal dataset considerato. La validazione in ambienti reali richiederebbe ulteriori raccolte dati, test su traffico live e confronto con scenari non presenti nel training set.

Un altro elemento importante è la differenza tra prestazioni predittive e operatività. Un IDS con alta accuracy può comunque generare falsi positivi o falsi negativi critici. In un contesto operativo, la soglia decisionale, la gestione degli alert e l'integrazione con procedure di risposta sono tanto importanti quanto il modello.

Capitolo 5 - Discussione e strategie di mitigazione

5.1 Interpretazione complessiva dei risultati

I risultati confermano che i modelli ensemble sono particolarmente efficaci sul dataset TON_IoT. LightGBM ottiene le migliori prestazioni complessive nella baseline binaria e multiclasse, con valori di accuracy, F1-score e Macro F1 molto elevati. Random Forest e XGBoost rimangono molto competitivi, mentre Logistic Regression mostra limiti più evidenti, soprattutto nel problema multiclasse.

Il risultato di LightGBM può essere spiegato dalla sua capacità di modellare relazioni non lineari tra feature tabulari e classi. Il traffico di rete contiene relazioni complesse tra porte, durata, pacchetti e byte; modelli basati su alberi e boosting possono catturare tali relazioni in modo più efficace rispetto a un modello lineare.

La differenza tra classificazione binaria e multiclasse è centrale. Nel caso binario, il modello deve stabilire se un campione sia normale o malevolo. Nel caso multiclasse, deve distinguere tra categorie specifiche, alcune delle quali possono condividere caratteristiche simili. Questo spiega perché il problema multiclasse sia più difficile e perché il Macro F1 diventi una metrica particolarmente rilevante.

5.2 Interpretabilità contro prestazioni

Un IDS deve essere accurato, ma anche interpretabile. In un ambiente operativo, un alert deve poter essere compreso, verificato e gestito. I modelli ensemble possono offrire prestazioni elevate, ma sono meno immediati da spiegare rispetto a Logistic Regression. L'analisi SHAP riduce parzialmente questo problema mostrando, con un criterio coerente, quali variabili incidono maggiormente sulle predizioni aggregate [10].

Logistic Regression è meno accurata, ma più semplice. I coefficienti del modello possono essere analizzati direttamente e l'inferenza è computazionalmente leggera. Questo la rende interessante in scenari embedded o didattici, come mostrato dal dimostratore Wokwi. Tuttavia, la semplicità ha un costo: nel dataset considerato, il modello lineare non raggiunge le prestazioni degli ensemble.

La scelta del modello deve quindi dipendere dal contesto. Se l'IDS opera su un server o gateway con risorse adeguate, LightGBM può essere preferibile. Se invece l'obiettivo è eseguire un modello molto compatto direttamente su un microcontrollore, può essere

necessario accettare un calo di prestazioni o ricorrere a tecniche specifiche di TinyML [19].

5.3 Limiti del dataset statico

TON_IoT è utile per confrontare modelli in modo controllato, ma rimane un dataset statico. In reti reali, il traffico cambia nel tempo. Nuovi dispositivi vengono aggiunti, firmware vengono aggiornati, configurazioni cambiano, utenti modificano comportamenti e nuovi attacchi emergono. Questo fenomeno, noto come concept drift, può ridurre le prestazioni di un modello addestrato su dati storici.

Un altro limite riguarda la rappresentatività. Un modello addestrato su un dataset può apprendere pattern specifici di quel dataset e non generalizzare perfettamente ad ambienti diversi. Per questo motivo, una valutazione più completa dovrebbe includere dataset multipli, traffico raccolto in laboratorio e test su scenari realistici.

La pipeline sviluppata rappresenta quindi una base sperimentale, non una soluzione definitiva. Il valore principale del lavoro è la riproducibilità: gli esperimenti possono essere rieseguiti, modificati e ampliati.

5.4 Limiti della simulazione Wokwi

Il dimostratore Wokwi è utile per mostrare un primo passaggio verso l'inferenza embedded, ma presenta limiti evidenti. La simulazione non misura consumo energetico reale, temperatura, interferenze, latenza di rete fisica o vincoli hardware completi. Inoltre, il modello embedded lavora su campioni statici esportati, non su traffico acquisito in tempo reale da un'interfaccia di rete.

Nonostante questi limiti, Wokwi è utile in fase di tesi perché consente di dimostrare il funzionamento del codice embedded senza richiedere hardware fisico. Il passo successivo sarebbe riprodurre l'esperimento su dispositivi reali, misurando memoria occupata, tempo di inferenza, stabilità e consumo energetico.

5.5 Strategie di mitigazione

Un IDS basato su machine learning deve essere inserito in una strategia più ampia di difesa. Non può sostituire le misure preventive, ma può integrarle fornendo capacità di rilevamento. Le principali strategie di mitigazione includono segmentazione della rete, privilegio minimo, aggiornamenti firmware, autenticazione forte, inventario dei

dispositivi, monitoraggio del traffico, anomaly detection, logging, alerting e principi zero trust [3], [17], [18].

La segmentazione riduce il rischio di movimento laterale. I dispositivi IoT dovrebbero essere collocati in VLAN o reti separate rispetto a sistemi critici, server e postazioni utente. Il principio del privilegio minimo limita le autorizzazioni concesse a dispositivi, account e servizi. L'autenticazione forte riduce il rischio di accessi non autorizzati, soprattutto su broker MQTT, dashboard e API.

Gli aggiornamenti firmware sono essenziali per correggere vulnerabilità note. Tuttavia, devono essere gestiti con attenzione, soprattutto in ambienti industriali dove l'interruzione del servizio può essere costosa. L'inventario dei dispositivi consente di sapere cosa è collegato alla rete e quali versioni firmware sono in uso. Senza inventario, non è possibile applicare una gestione efficace della sicurezza.

Il monitoraggio del traffico e il logging centralizzato permettono di rilevare anomalie e ricostruire incidenti. Un IDS può generare alert, ma tali alert devono essere integrati in un processo operativo: classificazione della severità, verifica, risposta, escalation e miglioramento delle regole o dei modelli.

5.6 Domande tecniche prevedibili

Perché LightGBM è risultato il modello migliore? LightGBM è un modello di gradient boosting efficiente su dati tabulari. Nel dataset utilizzato ha mostrato il miglior equilibrio tra accuracy, Macro F1 e tempi di training.

Perché SMOTE non migliora sempre? SMOTE genera campioni sintetici, ma non garantisce automaticamente un miglioramento. Se il modello gestisce già bene lo sbilanciamento o se i campioni sintetici non aggiungono informazione utile, il beneficio può essere nullo.

Perché Isolation Forest ha prestazioni inferiori? Isolation Forest è non supervisionato e non utilizza direttamente le etichette di attacco durante l'addestramento. I modelli supervisionati dispongono di più informazione e risultano avvantaggiati in un dataset etichettato.

Perché il modello Wokwi non è LightGBM? Il dimostratore Wokwi serve a mostrare un modello compatto esportabile in firmware. Logistic Regression è più semplice da

convertire in codice C++ minimale, mentre LightGBM richiederebbe una strategia di esportazione più complessa.

Il progetto è pronto per un deployment reale? No. Il progetto è una base sperimentale riproducibile. Un deployment reale richiederebbe test su traffico live, gestione del drift, pipeline di aggiornamento, integrazione con logging e alerting, hardening dell'ambiente e validazione su hardware fisico.

5.7 Possibile percorso verso una validazione su hardware reale

Il passaggio da simulazione a hardware reale richiede una sequenza di attività progressive. In primo luogo, occorre scegliere il dispositivo target, ad esempio un ESP32 o un gateway edge più potente. In secondo luogo, occorre stabilire quali feature siano effettivamente disponibili sul dispositivo. Un modello addestrato su feature di flusso di rete non può essere eseguito direttamente su un sensore se quel sensore non osserva tali feature.

In terzo luogo, è necessario definire come raccogliere i dati in tempo reale. Un dispositivo embedded potrebbe ricevere feature già calcolate da un gateway, oppure potrebbe calcolare solo indicatori semplici. In quarto luogo, occorre misurare latenza, memoria e consumo energetico. Queste misure sono fondamentali perché un modello accurato ma troppo costoso potrebbe non essere adatto al deployment.

Il dimostratore Wokwi copre solo una parte di questo percorso: mostra che una forma compatta del modello può essere eseguita in un ambiente simulato. La validazione su dispositivi reali richiederebbe esperimenti aggiuntivi in laboratorio, con misurazioni fisiche e traffico generato o raccolto in condizioni controllate.

5.8 Indicazioni operative per un IDS IoT

Un IDS IoT dovrebbe essere progettato come componente di una catena operativa. Il primo passo è l'inventario dei dispositivi: senza sapere quali dispositivi sono presenti, è impossibile definire comportamento atteso e priorità. Il secondo passo è la segmentazione: i dispositivi IoT dovrebbero comunicare solo con i servizi necessari. Il terzo passo è la raccolta dei log e dei flussi di rete.

Una volta raccolti i dati, il modello IDS può generare alert. Tuttavia, gli alert devono essere classificati per severità e associati a procedure di risposta. Un falso positivo può essere gestito con revisione manuale o correzione della soglia; un falso negativo

richiede analisi delle cause e aggiornamento del modello o delle regole. La sicurezza non è quindi un evento singolo, ma un processo continuo.

È inoltre necessario stabilire una procedura di aggiornamento. Un modello addestrato una volta può diventare obsoleto. Nuovi dispositivi e nuovi attacchi richiedono raccolta periodica di dati, rivalutazione delle metriche e rilascio controllato di nuove versioni del modello.

5.9 Considerazioni etiche e di gestione dei dati

Il monitoraggio del traffico IoT può coinvolgere dati sensibili. Anche quando il dataset usato in tesi è pubblico o scaricato per finalità di ricerca, in un contesto reale la raccolta di traffico deve rispettare principi di minimizzazione, protezione e controllo degli accessi. I dati raccolti da dispositivi domestici, sanitari o industriali possono rivelare abitudini, processi o informazioni riservate.

Un sistema IDS dovrebbe quindi limitare la raccolta alle feature necessarie, proteggere i log, applicare politiche di retention e impedire accessi non autorizzati. La sicurezza del sistema di rilevamento è essa stessa parte del problema: un IDS compromesso potrebbe esporre informazioni preziose o essere usato per nascondere attacchi.

Capitolo 6 - Conclusioni

La tesi ha affrontato il tema della sicurezza nei sistemi Internet of Things attraverso un percorso sperimentale basato su Intrusion Detection Systems e machine learning. Il lavoro ha prodotto un repository riproducibile, strutturato e documentato, capace di eseguire esperimenti di classificazione binaria e multiclasse sul dataset TON_IoT.

La baseline ha confrontato Random Forest, LightGBM, XGBoost e Logistic Regression. I risultati mostrano che i modelli ensemble, in particolare LightGBM, ottengono prestazioni molto elevate. Logistic Regression risulta meno accurata, ma è utile come modello semplice, interpretabile e adatto a una dimostrazione embedded. Gli esperimenti estesi con SMOTE, tuning, MLPClassifier e Isolation Forest hanno permesso di ampliare il confronto e di discutere limiti e vantaggi di approcci differenti.

L'analisi SHAP e le misure di deployment hanno esteso la valutazione oltre le metriche predittive. Il dimostratore Wokwi ha mostrato come un modello compatto possa essere esportato e simulato su ESP32, evidenziando il compromesso tra accuratezza e portabilità.

Il lavoro non deve essere interpretato come soluzione definitiva per la sicurezza IoT, ma come esperimento concluso entro un perimetro dichiarato. Estensioni indipendenti dal completamento della tesi includono test su dataset aggiuntivi, valutazione del concept drift, acquisizione di traffico live e riproduzione su hardware fisico. Quest'ultimo passaggio consentirebbe di misurare aspetti non osservabili in simulazione, come memoria reale, latenza effettiva, consumo energetico e stabilità del dispositivo.

In conclusione, la tesi dimostra che modelli di machine learning possono supportare efficacemente il rilevamento di intrusioni in ambienti IoT, purché siano inseriti in una strategia di sicurezza più ampia e valutati con attenzione rispetto a riproducibilità, interpretabilità, limiti del dataset e vincoli di deployment.

Bibliografia

- [1] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood e A. Anwar, "TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems", IEEE Access, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [2] UNSW Canberra, "The TON_IoT Datasets", <https://research.unsw.edu.au/projects/toniot-datasets>.
- [3] ENISA, "Baseline Security Recommendations for IoT in the context of Critical Information Infrastructures", 2017.
- [4] OASIS, "MQTT Version 5.0", OASIS Standard, 2019.
- [5] L. Breiman, "Random Forests", Machine Learning, vol. 45, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.
- [6] T. Chen e C. Guestrin, "XGBoost: A Scalable Tree Boosting System", Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, doi: 10.1145/2939672.2939785.
- [7] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", Advances in Neural Information Processing Systems 30, 2017.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall e W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique", Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.
- [9] F. T. Liu, K. M. Ting e Z.-H. Zhou, "Isolation Forest", 2008 Eighth IEEE International Conference on Data Mining, 2008, doi: 10.1109/ICDM.2008.17.
- [10] S. M. Lundberg e S.-I. Lee, "A Unified Approach to Interpreting Model Predictions", Advances in Neural Information Processing Systems 30, 2017.
- [11] Docker documentation, "Docker Compose", <https://docs.docker.com/compose>.
- [12] Wokwi Docs, "diagram.json File Format" e "Project Configuration", <https://docs.wokwi.com/>.

- [13] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets", *Sustainable Cities and Society*, vol. 72, art. 102994, 2021, doi: 10.1016/j.scs.2021.102994.
- [14] T. M. Booi, I. Chiscop, E. Meeuwissen, N. Moustafa e F. T. H. den Hartog, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets", *IEEE Internet of Things Journal*, vol. 9, n. 1, pp. 485-496, 2022, doi: 10.1109/JIOT.2021.3085194.
- [15] S. Sicari, A. Rizzardi, L. A. Grieco e A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead", *Computer Networks*, vol. 76, pp. 146-164, 2015, doi: 10.1016/j.comnet.2014.11.008.
- [16] B. B. Zarpelão, R. S. Miani, C. T. Kawakani e S. C. de Alvarenga, "A survey of intrusion detection in Internet of Things", *Journal of Network and Computer Applications*, vol. 84, pp. 25-37, 2017, doi: 10.1016/j.jnca.2017.02.009.
- [17] NIST, "Foundational Cybersecurity Activities for IoT Device Manufacturers", NISTIR 8259, 2020, doi: 10.6028/NIST.IR.8259.
- [18] S. Rose, O. Borchert, S. Mitchell e S. Connelly, "Zero Trust Architecture", NIST Special Publication 800-207, 2020, doi: 10.6028/NIST.SP.800-207.
- [19] P. Warden e D. Situnayake, "TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers", O'Reilly Media, 2019.
- [20] Scikit-learn Developers, "LogisticRegression", scikit-learn API Reference, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [21] Scikit-learn Developers, "MLPClassifier", scikit-learn API Reference, https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html.